NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 8: Objects, Classes, Attributes, and Methods

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Describe an entity. Make its behavior explicit.
Create instances. Test every important change.

About these notes

These notes expand the supplied 87-page presentation *Programming for Engineers: Week 8 — Objects, Classes, Attributes, and Methods*, by Yanwei Wang. The slides determine the topics, sequence, terminology, examples, and scope. The explanations turn short slide statements into reading material that you can use before class, during practice, and when revising.

Course information

Course: ENG 101 — Programming for Engineers, lecture section 2L, 6 ECTS credits.
Instructor: Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
Term: Fall 2026.
Week 8: Lecture 22 — Objects, Classes, and Data Attributes; Lecture 23 — Methods and Object Behavior; Lecture 24 — Reliable Engineering Objects.

Source-page references. Grey alignment lines refer to the page number in the supplied PDF, including its title and divider pages. This is not always the number printed in the slide footer. Appendix A maps all 87 source pages to the corresponding sections of these notes. The original slide PDF is cited as [1].

Reading code. Gold panels contain Python source code; the green comments explain meaning, units, execution order, and intended observations. Line numbers are for discussion and must not be typed. Grey panels contain output, not Python instructions. Source code that contains an intentional defect is explicitly labelled before it is shown.

Source examples versus worked additions. The slide examples retain their behavior. Comments, readable line wrapping, and prerequisite imports have been added. Activity answers, diagnostic observations, repairs, and completed workflows are labelled as worked solutions or extensions. They are not claimed to be additional official slide content. Optional reference topics remain optional.

The same name is used for successive teaching versions

The first `Sensor` stores three fields. A later version adds `report()`; the next version stores interval limits and adds query/update/report methods. They are successive examples, not three definitions to mix in one live notebook. Separate example files import the required version. `PressureSensor` is an independent class, not a subclass of `Sensor`; inheritance is deferred.

Running the examples. Extract the complete source package. Keep the `examples` folder intact, since scripts import the small class definitions stored beside them. Open one example in your Python editor and run it. An import such as `from sensor_models import Sensor` reuses the annotated definition printed in the notes; it does not install a package. The combined `ENG101_Week8_Annotated_Examples.py` is self-contained and runs working demonstrations without those separate files or keyboard input.

Assessment note

These notes are reading and practice materials, not a new assessment brief. Assignment 4 and Laboratory 3 are preparation topics in the deck. Its temperature dataset is explicitly illustrative. Moodle remains authoritative for tasks, deadlines, grading, collaboration, permitted assistance, and submission rules. No additional requirements are introduced here.

How to study Week 8

You already know that a function receives inputs and performs a named task. You also know that a dictionary keeps named pieces of information together. Week 8 connects these two ideas: an object can keep the information about one entity together with operations that use or change that information.

The aim is not to replace every function with a class. It is to recognize a useful *responsibility*: for example, one sensor has a name, a current reading, a unit, and rules for interpreting that reading. A class may make those relationships easier to express and test.

Lecture	Main question	What to draw or record
22	What information belongs to one particular entity?	A class definition and the separate attributes of two instances.
23	What does this method read, change, and return?	The receiver, state before/after, returned value, and printed output.
24	What happens when a proposed change is invalid?	The input contract, expected exception, preserved state, and test evidence.

A reading cycle that makes objects visible

Before running a sample, identify each construction call and draw a box for each new instance. Put its attributes inside the box. Draw names such as `original` and `backup` outside, with arrows to the objects they select. When a method is called, underline the receiver before the dot. Trace the assignments inside the method, then write its returned value separately from anything printed.

After execution, compare the result with your prediction. Change one input or boundary and repeat. For an update, test both the resulting state and the return value. For a failure, check both the exception and the state that remains. These are applications of Week 7 testing, not a separate kind of evidence.

Trace the program

For every method call, ask four questions: **Which instance? Which current attributes? Which changes? Which returned result?** A useful trace answers all four without relying on a vague statement such as “the object does something.”

Scope and prerequisites

You will reuse strings and conditions, functions and local variables, lists and aliasing, dictionaries, assertions, and exception handling. New syntax is deliberately limited to simple classes, initialization, attributes, and ordinary instance methods. Inheritance, class variables, getters/setters, and detailed information hiding belong to Session 9. Abstract classes, advanced class hierarchies, NumPy, and external testing frameworks are not required.

Chapters 1–3 follow the three lectures. Chapter 4 collects the slide reference material. Chapter 5 provides further self-checks and worked answers using the same ideas. The final glossary and source-page map support revision.

Engineering meaning remains your responsibility

A unit string is documentation, not a unit-conversion system. A method called `is_safe()` evaluates the stated teaching rule; it does not certify a device or process. Illustrative densities and operating limits are not certified design data. A passing test supports one specified behavior under selected inputs, not every possible input or the validity of a physical model.

Week 8 in one sentence

Describe an entity → Give it state and behavior → Create instances → Test every change. Keep data meaningful, behavior explicit, and evidence inspectable.

Contents

About these notes 2

How to study Week 8 3

1. Objects, Classes, and Data Attributes 7
1.1. Learning goals and the central question . . . 7
1.2. Warm-up: a familiar dictionary record . . . 7
1.3. Why introduce object-oriented programming? . . . 8
1.4. You have already used objects and methods . . . 8
1.5. Four terms that must stay distinct . . . 9
1.6. One class definition, several instances . . . 10
1.7. The smallest class and its limitation . . . 10
1.8. Initialization establishes the expected state . . . 11
1.9. Local parameters, attributes, and the receiver . . . 12
1.10. Create two sensors and inspect their attributes . . . 13
1.11. Trace state after each assignment . . . 14
1.12. Inspect the object's type and its field's type . . . 15
1.13. Choose between a dictionary and an instance . . . 15
1.14. Worked activity: a material data model . . . 16
1.15. Test initial state immediately . . . 17
1.16. Debugging clinic: local name or attribute? . . . 17
1.17. A first-class design checklist . . . 19
1.18. Exit ticket: one ideal resistor . . . 19

2. Methods and Object Behavior 20
2.1. Learning goals and the receiver warm-up . . . 20
2.2. A first method uses its receiver's fields . . . 20
2.3. Why the call does not explicitly supply self . . . 21
2.4. Read data, obtain a method, or call a method? . . . 22
2.5. Specify the interval sensor before implementing it . . . 22
2.6. Query, update, and report have different jobs . . . 23
2.7. Returned values are separate from printed output . . . 24
2.8. Boundary cases and a complete update trace . . . 25
2.9. Four method contracts to recognize . . . 26
2.10. Prediction activity: an old result or current state? . . . 27
2.11. A method reference is not a decision result . . . 27
2.12. Calculate with a material's stored density . . . 29
2.13. Use objects in familiar lists and loops . . . 30
2.14. Filter whole entities rather than isolated numbers . . . 30
2.15. Aliasing and deliberate snapshots . . . 31
2.16. Debugging the receiver in the definition and call . . . 33
2.17. Lecture 23 exit ticket and summary . . . 34

3. Reliable Engineering Objects 35
3.1. Learning goals: state has a contract too . . . 35
3.2. Warm-up: invalid data or an exceeded limit? . . . 35
3.3. Define one responsibility and its limits . . . 36
3.4. Write the input and stored-state contract . . . 36

3.5. Validate before committing a change . . . 37
3.6. An ordinary function can support a class . . . 37
3.7. The checked pressure class, assembled . . . 38
3.8. A high reading remains visible . . . 39
3.9. Test a normal value, the boundary, and retained data . . . 40
3.10. A failed update needs two checks . . . 40
3.11. Check independence with a second object . . . 42
3.12. Combine objects, loops, and exception handling . . . 43
3.13. Build an object-oriented test matrix . . . 44
3.14. Public attributes can bypass the checked route . . . 45
3.15. Choose a class only when it improves the design . . . 46
3.16. Assignment 4 preparation and Laboratory 3 practice . . . 46
3.17. Review the design and answer the exit ticket . . . 48

4. Optional Reference and Debugging Notes **49**
4.1. The complete checked-number helper . . . 49
4.2. Each instance can own a fresh list . . . 51
4.3. A supplied list may still be shared . . . 52
4.4. Worked answer: an ideal resistor . . . 54
4.5. Common class mistakes: read the cause, not only the symptom . . . 56

5. Synthesis and Cumulative Practice **58**
5.1. Connect the ideas before memorizing the syntax . . . 58
5.2. Quick reference for reading unfamiliar class code . . . 58
5.3. Twelve practice questions . . . 59
5.4. Worked answers and explanations . . . 60
5.5. Readiness checklist and the next scope boundary . . . 62

6. Glossary and Revision Checklist **63**
6.1. Week 8 glossary . . . 63
6.2. A one-page review procedure . . . 65

A. Sources, Clarifications, and Slide Coverage **66**
A.1. Source basis and suggested reading . . . 66
A.2. Adaptation and clarification record . . . 66
A.3. Complete source-page coverage map . . . 67

Bibliography **70**

1 Objects, Classes, and Data Attributes

Lecture-slide alignment: Week 8 source PDF, pp. 4–28. PDF page numbers include title and divider pages.

1.1. Learning goals and the central question

The central question of Lecture 22 is: *what information belongs to one particular engineering entity?* A sensor reading is more than a number. Its identity and unit are also needed before the number can be interpreted. A class gives us one way to organize that information consistently.

By the end of the lecture, you should be able to explain when grouping data and behavior is useful; distinguish a class, an instance, an attribute, and a method; initialize a small class with `__init__`; explain `self`; create two instances with separate data; and inspect, modify, and test a simple object. The lecture develops syntax gradually so that each new feature solves a visible problem.

Key idea

A class is useful when its structure makes an engineering responsibility clearer. Creating a class is not automatically an improvement over a dictionary and a well-designed function.

1.2. Warm-up: a familiar dictionary record

Lecture-slide alignment: Week 8 source PDF, p. 6. PDF page numbers include title and divider pages.

The opening example deliberately uses no new class. A dictionary stores three named pieces of information: a name, a unit, and a current value. An ordinary function receives that dictionary and returns a report string. Read this program as a reminder of the tools that already work.

Slide example. Source PDF page(s) 6.

```
# ENG 101 Week 8: A familiar sensor record
# Basis: source PDF page(s) 6; slide example.
# Store named fields in the familiar Week 6 dictionary form.
sensor = {"name": "Tank temperature", "unit": "deg C", "value": 82.5}

def format_reading(record):
    # The dictionary is supplied explicitly to this ordinary function.
    return f"{record['name']}: {record['value']:.1f} {record['unit']}"

# Return the string first, then display it with print().
print(format_reading(sensor))
```

Code 1.1: A familiar sensor record

Example file: `ex01_dictionary_bridge.py`

```
Tank temperature: 82.5 deg C
```

The dictionary variable `sensor` refers to a record whose `"value"` is the floating-point number `82.5`. The unit is a separate string. In `format_reading(record)`, the parameter `record` refers

to the dictionary supplied by the caller. The function looks up its fields using keys, formats the number to one decimal place, and returns the assembled text. The outer `print()` displays that returned value.

Warm-up answers

The stored data are the name `"Tank temperature"`, the unit `"deg C"`, and the value `82.5`. The operation is formatting a human-readable reading. The sensor record is supplied explicitly in `format_reading(sensor)`. The returned string is `Tank temperature: 82.5 deg C`.

Nothing in this example is obsolete. Later, a call written as `sensor.report()` will select the sensor through the expression before the dot. The information still needs to reach the operation; the way we express that relationship will change.

1.3. Why introduce object-oriented programming?

Lecture-slide alignment: Week 8 source PDF, p. 7. PDF page numbers include title and divider pages.

Imagine a laboratory with several sensors. Each sensor has its own name, unit, and reading. Each also needs operations such as preparing a report or checking a reading against limits. Maintaining the relationships becomes easier when one structure makes both the data and the relevant operations visible.

Object-oriented programming, abbreviated OOP, is introduced here as organizing a program around such entities. The class describes the kind of entity and the operations available for it. Each instance represents one particular entity with its own current data. One definition can therefore support many sensors without copying the method definitions for every sensor in the source code.

This does not mean that every physical noun deserves a class. A single unit conversion may be clearer as a function. A small collection of named data may be clearer as a dictionary. The reason to introduce a class is to clarify how state and related operations belong together, not merely to use a recently learned keyword.

1.4. You have already used objects and methods

Lecture-slide alignment: Week 8 source PDF, p. 8. PDF page numbers include title and divider pages.

A Python string and a Python list are already objects. Their types provide operations that you have used in previous weeks. The next example contrasts a method that returns a new string, a method that changes a list, and an ordinary function call.

Slide example. Source PDF page(s) 8.

```python
# ENG 101 Week 8: Objects and methods already encountered
# Basis: source PDF page(s) 8; slide example.
# Create a string and a list; both are objects of existing Python types.
label = " tank "
readings = [82.5, 85.0]
print(label.strip())        # Display new text, not a changed label.
readings.append(90.0)       # Mutate this list by appending one number.
print(len(readings))        # Call a function to count three elements.
print(readings)             # Display the list after the mutation.
```

Code 1.2: Objects and methods already encountered

Example file: `ex02_familiar_methods.py`

```
tank
3
[82.5, 85.0, 90.0]
```

In `label.strip()`, `label` selects the string on which the method operates. The method returns cleaned text. It does not reassign `label`, so the original variable still refers to the string with surrounding spaces. By contrast, `readings.append(90.0)` changes the existing list. Its return value is `None`; the important effect is the list mutation.

The call `len(readings)` passes a list to a function rather than using dot-call syntax. All three operations are legitimate tools. Week 8 does not require you to rewrite built-in functions as methods. It teaches you to define a new type whose methods express the operations of a chosen engineering entity.

Stop, predict, and try it

Predict `label` after the program finishes. Then ask which line changed `readings`. The two answers should refer to the effect of an operation, not merely to whether its syntax contains a dot.

1.5. Four terms that must stay distinct

Lecture-slide alignment: Week 8 source PDF, p. 9. PDF page numbers include title and divider pages.

Term	Meaning in these notes	Example
Class	A definition of an object type and its operations.	`Sensor`
Instance / object	One particular object created from a class.	The object named `temperature`
Data attribute	A named value associated with an instance.	`temperature.value`
Method	An operation called using an instance.	`temperature.report()`

State is the current stored information. For a basic sensor, that includes its name, unit, and reading. **Behavior** is what the object can calculate, answer, or change. A report is behavior that reads state and returns text. An update is behavior that changes state.

There is also a naming distinction. Strictly speaking, `temperature` is a variable name referring to an instance; it is not the class definition. Saying "the temperature object" is convenient shorthand, but drawing an arrow from the name to the object helps when a second name later refers to the same object.

1.6. One class definition, several instances

Lecture-slide alignment: Week 8 source PDF, p. 10. PDF page numbers include title and divider pages.

The slide's diagram connects one `Sensor` class to two instances. The temperature instance contains `"Tank temperature"`, `"deg C"`, and `82.5`. The pressure instance contains `"Pipe pressure"`, `"kPa"`, and `315.0`. They share a type, not one shared reading.

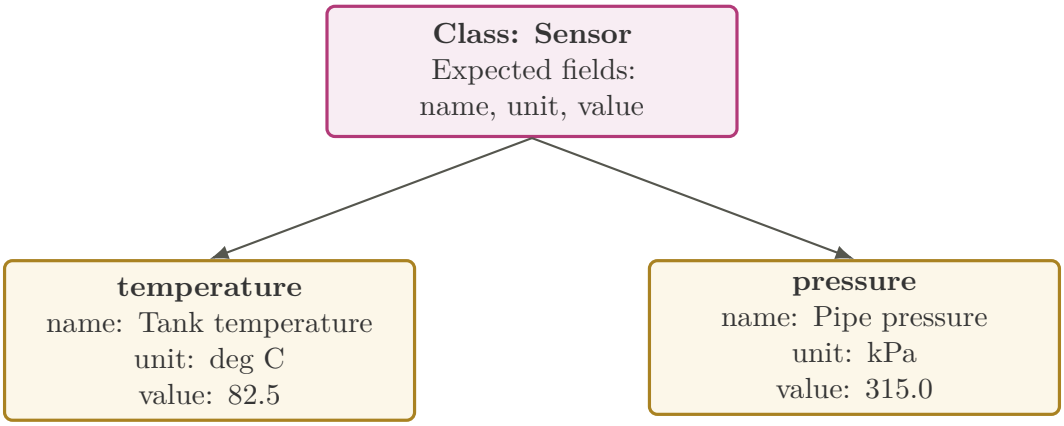


Figure 1.1.: One definition describes the structure; separate instances carry different current data. Adapted from source PDF page 10.

The engineering analogy is one design description and two specific devices. Do not push that analogy too far: these Python objects are software records. Creating them does not connect to a physical temperature or pressure sensor.

1.7. The smallest class and its limitation

Lecture-slide alignment: Week 8 source PDF, pp. 11–12. PDF page numbers include title and divider pages.

The statement `class Sensor:` begins a class definition. It requires an indented body. The `pass` statement supplies a body that deliberately performs no operation. Calling `Sensor()` then creates an instance of this simple class.

Slide example with an added observation. Source PDF page(s) 11.

```
# ENG 101 Week 8: Start with an empty class
# Basis: source PDF page(s) 11; slide example with an added observation.
# A placeholder class demonstrates syntax, not the final design.
class Sensor:
    pass  # Python requires a body; pass performs no operation.

s1 = Sensor()                    # Create one instance.
s1.name = "Tank temperature"     # Attach each field manually.
s1.unit = "deg C"
s1.value = 82.5
# Added observation: inspect all three fields after assigning them.
print(s1.name, s1.value, s1.unit)
```

Code 1.3: Start with an empty class

Example file: `ex03_empty_class.py`

```
Tank temperature 82.5 deg C
```

Each assignment after construction creates an attribute on `s1`. The expression on the left of `=` tells you which attribute is assigned. For example, `s1.unit = "deg C"` assigns the field `unit` on this instance; it does not create a general variable named `unit` in every sensor.

Manual assignment is useful for understanding syntax but fragile as a repeated workflow. A second instance can be created without receiving all the same fields. Reading a missing field raises `AttributeError`. The problem is not that Python forgot information: the program never established that information on the selected instance.

Design principle

Decide which attributes must exist before an object is used. When every instance requires the same fields, establish them in one repeatable initialization step instead of relying on several later manual assignments.

1.8. Initialization establishes the expected state

Lecture-slide alignment: Week 8 source PDF, pp. 13–14. PDF page numbers include title and divider pages.

The special method `__init__` is the initialization step used in this lecture. Its name has two underscores before and after `init`. When the class is called in the usual way, Python creates an instance and invokes this method to establish its initial state.

```python
# Week 8, source PDF page 13: the first initialized Sensor.
class Sensor:
    """Store a sensor name, its unit, and its current value."""

    def __init__(self, name, unit, value):
        # self identifies the one instance being initialized.
        self.name = name       # Keep the caller's human-readable label.
        self.unit = unit       # Document the unit; no conversion occurs.
        self.value = value     # Store the supplied current reading.
```

Code 1.4: The first initialized Sensor: three data attributes

Complete definition in `examples/sensor_basics.py`.

Read the header as a function definition inside the class. `self` receives the particular new instance. The remaining parameters receive the arguments supplied in the construction call. Each assignment stores one supplied value on that instance.

A call such as `Sensor("Tank temperature", "deg C", 82.5)` therefore gives `name` the string `"Tank temperature"`, `unit` the string `"deg C"`, and `value` the number `82.5`. No caller argument is written for `self` in this ordinary construction call.

Stage	Event in this example
1	Evaluate the arguments in the `Sensor(...)` call.
2	Create a new instance of the class.
3	Run `__init__` with that instance and the supplied values.
4	After successful initialization, give the caller the instance.

Initialization is not a value-returning factory function

Do not write `return self` or return another non-`None` value from `__init__`. The class call gives the caller the instance; `__init__` initializes it. A non-`None` return from the initializer causes `TypeError`. This distinction is explicit on source page 14; see also the narrowly scoped technical reference [6].

The short class assumes its inputs are appropriate. Storing a field consistently is not the same as validating its type, unit, or physical meaning. Those checks are developed later, rather than being silently attributed to this first initializer.

1.9. Local parameters, attributes, and the receiver

Lecture-slide alignment: Week 8 source PDF, pp. 15–16. PDF page numbers include title and divider pages.

The statement `self.value = value` contains two different references. On the right, `value` is a local parameter name used during the method call. On the left, `self.value` selects an attribute on the receiver. The assignment stores the parameter's current value as that attribute.

The matching spelling is a convention for readability, not a requirement. A parameter named `initial_value` could equally be stored with `self.value = initial_value`. What matters is the assignment target. Assigning only `value = initial_value` would leave a local name instead of creating the instance attribute.

Receiver means the instance selected for a method call. In `temperature.report()`, the receiver is the instance referred to by `temperature`. In `pressure.report()`, it is the pressure instance. Inside each call, `self` refers to that call's receiver. The same method definition can therefore read different values on different calls.

What self does and does not mean

`self` is the conventional name of the first parameter of an ordinary instance method. It is not a special Python keyword, a reference to all instances, or a request to make a copy. It identifies the instance used by this call. The report calls on source page 16 preview behavior introduced in Lecture 23; the three-field class alone does not yet define `report()`.

An attribute remains accessible through the object after initialization has finished. The local parameter name does not become a variable available to the caller. The important persistence is the relationship established on the instance, not the lifetime of the temporary local name.

1.10. Create two sensors and inspect their attributes

Lecture-slide alignment: Week 8 source PDF, pp. 17–18. PDF page numbers include title and divider pages.

The script imports the three-field class from `sensor_basics.py`. Two construction calls then create the instances, and dot notation reads their attributes.

Slide example. Source PDF page(s) 13, 17.

```
# ENG 101 Week 8: Two instances of the same class
# Basis: source PDF page(s) 13, 17; slide example.
# Reuse the exact three-field class printed earlier in these notes.
from sensor_basics import Sensor

temperature = Sensor("Tank temperature", "deg C", 82.5)
pressure = Sensor("Pipe pressure", "kPa", 315.0)
print(temperature.name)                      # Read this sensor's label.
print(temperature.value, temperature.unit)   # Read its value and unit.
print(pressure.value, pressure.unit)         # Read the other instance.
```

Code 1.5: Two instances of the same class

Example file: `ex04_create_two.py`

```
Tank temperature
82.5 deg C
315.0 kPa
```

The two calls use the same parameter order but different arguments. Reading `temperature.value` selects the temperature instance's current value. Reading `pressure.value` selects the pressure instance's current value. Dot notation makes the owning instance visible at the use site.

An assignment such as `temperature.value = 85.0` replaces that instance's current reading. It does not run a validation method in this plain class, and it does not update `pressure.value`. A later read observes the newly stored value.

Engineering check

The field `unit` is text. Changing `temperature.unit` from `"deg C"` to another label does not convert the number. The data, label, and intended physical quantity must remain consistent. The class cannot infer the missing conversion merely because the label changed.

1.11. Trace state after each assignment

Lecture-slide alignment: Week 8 source PDF, pp. 19–20. PDF page numbers include title and divider pages.

This activity combines instance attributes with the Week 1 distinction between a name and a stored value. Before executing it, write the three final numbers on paper.

Slide example. Source PDF page(s) 19-20.

```
# ENG 101 Week 8: A local name is not a live link
# Basis: source PDF page(s) 19-20; slide example.
from sensor_basics import Sensor  # The three-field class from page 13.

# Two construction calls make separate instances, even with equal values.
a = Sensor("A", "deg C", 20.0)
b = Sensor("B", "deg C", 20.0)
a.value = 25.0                # Assign an attribute on a only.
local_value = a.value         # Read the number currently stored in a.
local_value = 99.0            # Rebind this name, not a.value.
print(a.value, b.value, local_value)  # Predict 25.0, 20.0, and 99.0.
```

Code 1.6: A local name is not a live link

Example file: `ex05_state_trace.py`

```
25.0 20.0 99.0
```

Statement completed	a.value	b.value	local_value
Create `a` and `b`	20.0	20.0	Not assigned
`a.value = 25.0`	25.0	20.0	Not assigned
`local_value = a.value`	25.0	20.0	25.0
`local_value = 99.0`	25.0	20.0	99.0

The first change targets an attribute on `a`. The last change targets only the name `local_value`. Reading a numeric attribute does not create a live link that redirects later assignments back to the object. The final output is therefore `25.0 20.0 99.0`.

This example uses immutable numbers. Later examples consider aliases of whole mutable objects and mutable list attributes. Do not generalize the result to mean that all assigned data are copied: the correct rule is to follow which names refer to which objects and which assignment target is changed.

1.12. Inspect the object's type and its field's type

Lecture-slide alignment: Week 8 source PDF, p. 21. PDF page numbers include title and divider pages.

An object and a value stored inside it need not have the same type. The sensor has type `Sensor`; its current reading in this example has type `float`.

Slide example. Source PDF page(s) 21.

```python
# ENG 101 Week 8: Inspect an instance and one attribute
# Basis: source PDF page(s) 21; slide example.
from sensor_basics import Sensor  # Reuse the first initialized class.

s1 = Sensor("Tank temperature", "deg C", 82.5)
print(type(s1).__name__)          # Read the name of the object's type.
print(isinstance(s1, Sensor))     # Check the type of the whole object.
print(isinstance(s1.value, float))  # Check the stored number's type.
```

Code 1.7: Inspect an instance and one attribute

Example file: `ex06_type_checks.py`

```
Sensor
True
True
```

`type(s1)` returns the object's type. Reading its `__name__` attribute produces the name `Sensor`. The function `isinstance(s1, Sensor)` asks whether the object has the expected type relationship. The final call asks a different question about `s1.value`, not about the whole sensor.

Type checks are evidence about representation. They do not establish that a number has the intended unit or that a sensor label corresponds to a real device. That additional meaning still belongs in the contract and the data source.

1.13. Choose between a dictionary and an instance

Lecture-slide alignment: Week 8 source PDF, pp. 22–23. PDF page numbers include title and divider pages.

Task	Dictionary record	Instance
Read a value	`record["value"]`	`sensor.value`
Change a value	`record["value"] = x`	`sensor.value = x`
Request a report	`report(record)`	`sensor.report()`
Reason to choose	Simple named data with external functions.	A meaningful entity with related operations.

A class can keep related state and operations together, provide a common reusable interface, support tests of individual entities, and make a program read like an engineering procedure. These are potential advantages, not guaranteed outcomes.

There are costs: more syntax and names, changes to state that may be harder to notice, and aliases that can expose unintended changes. A class with an unclear responsibility adds complexity rather than removing it. For a one-off calculation with explicit inputs and no persistent entity, a function can remain the clearer choice.

Design principle

Choose the representation that makes responsibility visible. Do not convert every dictionary into a class merely because classes are now available.

1.14. Worked activity: a material data model

Lecture-slide alignment: Week 8 source PDF, p. 24. PDF page numbers include title and divider pages.

The material activity asks for a name and a constant density in kg/m^3. Its two illustrative densities are 2700 for aluminum and 7850 for steel. The task is to define the class, initialize its fields, and create and inspect two instances.

Worked activity solution. Source PDF page(s) 24.

```
# ENG 101 Week 8: Create two material records
# Basis: source PDF page(s) 24; worked activity solution.
# Worked completion of the slide's class-and-instance activity.
class Material:
    """Store a name and illustrative constant density in kg/m^3."""

    def __init__(self, name, density_kg_m3):
        self.name = name                    # Identity of this material.
        self.density_kg_m3 = density_kg_m3  # Assume positive finite input.

# Illustrative values from the slide; not certified design properties.
aluminum = Material("Aluminum", 2700.0)
steel = Material("Steel", 7850.0)
print(aluminum.name, aluminum.density_kg_m3, "kg/m^3")
print(steel.name, steel.density_kg_m3, "kg/m^3")
```

Code 1.8: Create two material records

Example file: `ex07_material_record.py`

```
Aluminum 2700.0 kg/m^3
Steel 7850.0 kg/m^3
```

The attribute name `density_kg_m3` makes the quantity and unit visible. The initializer stores the value supplied by the caller, and the two objects retain different densities. This version has no calculation method; Lecture 23 will use the stored density to calculate a mass from a supplied volume.

The code assumes a positive finite density. It does not check that condition, and the data are illustrative programming inputs, not certified material-property values. Adding a comment about an assumption helps a reader understand the intended domain but does not make Python enforce it.

1.15. Test initial state immediately

Lecture-slide alignment: Week 8 source PDF, p. 25. PDF page numbers include title and divider pages.

An initialization test compares the state visible after construction with the values promised by the interface. Start with a tiny case whose expected fields are easy to inspect.

Slide example. Source PDF page(s) 25.

```
# ENG 101 Week 8: Inspect initial state immediately
# Basis: source PDF page(s) 25; slide example.
from sensor_basics import Sensor  # Use the three-field Sensor class.

s = Sensor("Test", "deg C", 20.0)  # Supply known initialization values.
assert s.name == "Test"           # Check the label.
assert s.unit == "deg C"          # Check the documented unit.
assert s.value == 20.0            # Check the initialized reading.
print("Initialization checks passed")  # Reached only if checks pass.
```

Code 1.9: Inspect initial state immediately

Example file: `ex08_initialization_tests.py`

```
Initialization checks passed
```

Each assertion checks one field. If an assertion is false, execution stops with `AssertionError`; the success message is reached only after all three comparisons pass. The numerical equality is appropriate here because the test checks the value directly assigned, not a long floating-point calculation with accumulated rounding.

These checks do not validate a real instrument, its calibration, or every possible input. They show that this class establishes the expected state for this selected case. Creating a second instance with different arguments is the natural next test because a shared-state mistake may not be visible with only one object.

1.16. Debugging clinic: local name or attribute?

Lecture-slide alignment: Week 8 source PDF, p. 26. PDF page numbers include title and divider pages.

This example intentionally raises an error. Run it separately from the working demonstrations.

Intentional error; run separately. Slide example. Source PDF page(s) 26.

```python
# ENG 101 Week 8: Intentional local-name error
# Basis: source PDF page(s) 26; slide example.
# Intentional defect: the initializer creates only a local name.
class Sensor:
    def __init__(self, initial_value):
        value = initial_value  # BUG: no assignment to self.value.

s = Sensor(82.5)       # The initializer runs but does not store the field.
print(s.value)        # Expected AttributeError: this field is absent.
```

Code 1.10: Intentional local-name error

Example file: `intentional_errors/ex09_local_not_attribute.py`

```
AttributeError: 'Sensor' object has no attribute 'value'
```

Only the final exception line is shown; traceback paths and version-dependent wording are omitted.

The initializer *does* run. Its argument is received as `initial_value`, then assigned to the local name `value`. But no statement assigns `self.value`, so the attribute requested by `s.value` does not exist. The smallest repair changes the assignment target.

Worked repair. Source PDF page(s) 26.

```python
# ENG 101 Week 8: Repair the initialization assignment
# Basis: source PDF page(s) 26; worked repair.
# Worked repair: store the input on the receiver, not only locally.
class Sensor:
    def __init__(self, initial_value):
        self.value = initial_value  # Establish a persistent attribute.

s = Sensor(82.5)              # Initialize the expected field.
assert s.value == 82.5       # Retain a test for the original defect.
print(s.value)               # This attribute now exists.
```

Code 1.11: Repair the initialization assignment

Example file: `ex10_attribute_repair.py`

```
82.5
```

Diagnostic answers

The initialization call succeeds in the faulty version; it simply fails to establish the intended field. The missing field is exposed only when the caller reads it. The repair is `self.value = initial_value`, followed by a regression check that the field contains `82.5`. Replacing the attribute read with a guessed constant would hide the symptom rather than repair the data model.

1.17. A first-class design checklist

Lecture-slide alignment: Week 8 source PDF, p. 27. PDF page numbers include title and divider pages.

A first class should have one clear engineering responsibility and meaningful class and instance names. Its attributes should have documented meanings and units. Its initializer should establish the state required before use. Two distinct construction calls should be used when checking that instances do not unintentionally share data.

The final check is observational: what should a caller see after construction or an update? "The program ran" is not a specification. "The second sensor retained its original reading after the first sensor changed" is a claim that can be tested.

1.18. Exit ticket: one ideal resistor

Lecture-slide alignment: Week 8 source PDF, p. 28. PDF page numbers include title and divider pages.

The exit ticket requests a class for an ideal resistor, one resistance attribute in ohms, an initializer, and two instances with resistances of 4 and 8 ohms. The following answer deliberately stays at the data-model stage.

Worked exit-ticket solution. Source PDF page(s) 28.

```
# ENG 101 Week 8: Lecture 22 resistor exit ticket
# Basis: source PDF page(s) 28; worked exit-ticket solution.
# Worked answer limited to the data model requested in Lecture 22.
class Resistor:
    def __init__(self, resistance_ohm):
        # This first record assumes the caller supplies a valid value.
        self.resistance_ohm = resistance_ohm

first = Resistor(4.0)          # One ideal resistor, resistance in ohms.
second = Resistor(8.0)         # A separate instance of the same class.
print(first.resistance_ohm, second.resistance_ohm)
print(first is second)        # Equal class does not mean same identity.
```

Code 1.12: Lecture 22 resistor exit ticket

Example file: `ex11_resistor_record.py`

```
4.0 8.0
False
```

Each initializer receives a different new instance as `self`; therefore `first is second` is false. The two resistance attributes contain the separately supplied arguments.

This record assumes valid resistance. The optional answer later adds a positivity check and a current calculation; this first version does not yet reject invalid data.

Lecture 22 summary. A class defines a type; attributes store each instance's state. Initialization establishes expected fields. Draw the instances, trace assignment targets, and test what the caller should observe.

2 Methods and Object Behavior

Lecture-slide alignment: Week 8 source PDF, pp. 29–53. PDF page numbers include title and divider pages.

2.1. Learning goals and the receiver warm-up

Lecture-slide alignment: Week 8 source PDF, pp. 29–31. PDF page numbers include title and divider pages.

Lecture 23 gives objects useful behavior. You will define and call ordinary instance methods, explain the automatically supplied receiver, distinguish calculation/query/update/report contracts, trace results and state changes, and process lists of instances. You will also diagnose missing calls, receiver mistakes, and aliasing.

Begin with the expressions from the warm-up. In `label.strip()`, the receiver is the string named `label`; the result is a cleaned string. In `readings.append(90.0)`, the receiver is the list named `readings`; the list changes and the return value is `None`. In the next `Sensor` class, `sensor.report()` has the sensor instance as receiver and returns text.

Key idea

The receiver and the returned result are separate questions. Selecting an object tells the method *which entity* to use. The method's contract tells the caller whether state changes and what value comes back.

2.2. A first method uses its receiver's fields

Lecture-slide alignment: Week 8 source PDF, p. 32. PDF page numbers include title and divider pages.

The first reporting class retains the three attributes from Lecture 22 and adds a method named `report`. Its body reads the attributes of `self` and returns a string.

```python
# Week 8, source PDF page 32: add reporting to the three-field class.
class Sensor:
    """Store one reading and return a report without printing it."""

    def __init__(self, name, unit, value):
        # These attributes remain accessible after initialization ends.
        self.name = name
        self.unit = unit
        self.value = value

    def report(self):
        # Read the receiver's current fields; format one decimal place.
        # Returning text leaves the display decision with the caller.
        return f"{self.name}: {self.value:.1f} {self.unit}"
```

Code 2.1: The three-field Sensor with a report method

Complete definition in `examples/reporting_sensor.py`.

The method definition is indented inside the class, at the same level as `__init__`. Its body is

indented one level further. The parameter `self` supplies access to the selected instance. Thus `self.name` is not a search for some unrelated global sensor name: it reads the name stored on the receiver.

The expression `self.value:.1f` inside the f-string formats the numeric value to one decimal place. Formatting changes the text used for display; it does not change the stored number. The method contains no assignment to the instance and no `print()`, so it neither changes the state nor displays a message by itself.

The return value allows the caller to decide whether to print the report immediately, store it for later, or compare it with expected text in a test. This continues Week 4's separation between producing a result and displaying it.

2.3. Why the call does not explicitly supply self

Lecture-slide alignment: Week 8 source PDF, p. 33. PDF page numbers include title and divider pages.

The ordinary call `s.report()` supplies the object named `s` as the first argument to the method. For this specific ordinary method, it has the same effect as `Sensor.report(s)`.

Slide example. Source PDF page(s) 32-33.

```
# ENG 101 Week 8: Explain the automatically supplied receiver
# Basis: source PDF page(s) 32-33; slide example.
# This module defines the three-field Sensor with a report method.
from reporting_sensor import Sensor

s = Sensor("Tank temperature", "deg C", 82.5)
print(s.report())            # The receiver s is supplied automatically.
print(Sensor.report(s))      # Equivalent for this ordinary method.
```

Code 2.2: Explain the automatically supplied receiver

Example file: `ex12_receiver_binding.py`

```
Tank temperature: 82.5 deg C
Tank temperature: 82.5 deg C
```

Both calls return the same text because both select the same instance and run the same operation. The first is the normal everyday syntax. The explicit class call is shown to explain binding, not to encourage a more cumbersome calling style.

Trace an ordinary bound method call

In `s.report()`, first identify `s`. The method call then makes that instance available as `self`. The body reads `self.name`, `self.value`, and `self.unit`; constructs a string; and returns it to the caller. The surrounding `print()` displays the result. No new sensor is constructed during this report call.

This explanation is limited to the ordinary instance methods taught in the deck. Other forms of class behavior are outside the week's scope. The source explicitly uses the equivalence only for this case; the Python tutorial explains the same binding rule [5].

2.4. Read data, obtain a method, or call a method?

Lecture-slide alignment: Week 8 source PDF, p. 34. PDF page numbers include title and divider pages.

Expression	Meaning in the current class
`s.value`	Read the stored numeric value.
`s.report`	Obtain a method object without executing its body.
`s.report()`	Execute the method and obtain its returned string.
`s.value()`	Attempt to call a number; this produces a type error here.

Parentheses are not decoration. A method can be obtained as a value without being called, but that value is not the computed report or Boolean answer. This is the same distinction you encountered when writing `raw.strip` instead of `raw.strip()` during string processing.

Likewise, putting call parentheses after an ordinary numeric attribute does not turn it into a calculation. The deliberate error in Example `ex42` demonstrates that a stored float is not callable; its annotated listing appears in the optional debugging reference. The ordinary action is to read `s.value` without parentheses.

2.5. Specify the interval sensor before implementing it

Lecture-slide alignment: Week 8 source PDF, pp. 35–36. PDF page numbers include title and divider pages.

The next version of `Sensor` has five attributes: `name`, `unit`, `value`, `lower_limit`, and `upper_limit`. The first two identify the measurement. The third is the current reading. The last two specify the inclusive acceptable interval.

Interval-sensor teaching contract

The caller supplies a finite numeric reading and ordered numeric limits in the same unit. The query returns true when

$$\text{lower limit} \leq \text{value} \leq \text{upper limit}.$$

Updating replaces the current reading. Reporting returns text. The first implementation assumes valid input rather than checking it. “Safe” in the method name means only within the stated teaching limits.

The call `Sensor("Tank", "deg C", 82.5, 0.0, 90.0)` makes the argument order visible: label, unit, reading, lower boundary, upper boundary. The reading can change while the boundaries remain unchanged. A method that updates only the reading should not accidentally move the operating limits at the same time.

The following listing assembles the initializer and three methods displayed separately on source pages 36–39. The method behavior is preserved; comments explain the complete class in one place.

```python
class Sensor:
    """Bundle a current reading, units, limits, and related operations.

    Assume finite numeric readings, ordered limits, and matching units.
    is_safe() means within stated limits, not certified physical safety.
    """

    def __init__(self, name, unit, value, lower_limit, upper_limit):
        # Initialize fields on this instance, not on another sensor.
        self.name = name                    # Human-readable identity.
        self.unit = unit                    # Unit label, not a converter.
        self.value = value                  # The current measurement.
        self.lower_limit = lower_limit      # Inclusive lower boundary.
        self.upper_limit = upper_limit      # Inclusive upper boundary.

    def is_safe(self):
        """Return whether the reading is within the inclusive limits."""
        # A query evaluates the stored value now without changing it.
        return self.lower_limit <= self.value <= self.upper_limit

    def update(self, new_value):
        """Replace the reading; assume valid numeric input."""
        self.value = new_value              # Replace the current value.
        # No explicit return statement: the call returns None.

    def report(self):
        """Return a labeled result without changing the stored reading."""
        # Call the query; parentheses are necessary to evaluate it.
        if self.is_safe():
            status = "WITHIN LIMITS"
        else:
            status = "OUTSIDE LIMITS"
        # Adjacent f-strings form one message; the caller prints it.
        return (f"{self.name}: {self.value:.1f} {self.unit} | "
                f"{status}")
```

Code 2.3: Annotated definition of `Sensor`

Extract from `examples/sensor_models.py`; use that complete file when running the examples.

2.6. Query, update, and report have different jobs

Lecture-slide alignment: Week 8 source PDF, pp. 37–39. PDF page numbers include title and divider pages.

2.6.1. The query evaluates an inclusive rule

`is_safe()` returns the result of a chained comparison. If the current reading is exactly the lower or upper limit, the result is true. If it lies outside either boundary, the result is false. The query reads three attributes and returns a Boolean; it does not change a measurement.

The name `is_safe` is retained from the source example. Its guarantee is deliberately narrower than that name might suggest: it checks a stated interval, not calibration, sensor accuracy, hardware status, or complete process safety. The report labels `WITHIN LIMITS` and `OUTSIDE LIMITS` make that narrower meaning more explicit.

2.6.2. The update replaces stored state

`update(new_value)` assigns the new value to `self.value`. The previous reading is replaced; this class does not keep measurement history. The successful call returns `None`, because no value-returning statement is executed.

Consequently, `s.update(95.0)` is an update, whereas `s = s.update(95.0)` would both update the object and then rebind `s` to `None`. These are two separate effects. A program can lose its convenient reference to the sensor even though the mutation itself succeeded.

2.6.3. The report uses the query rather than duplicating it

Inside `report()`, `self.is_safe()` evaluates the current rule. A local variable `status` receives the appropriate string, and the method returns the reading plus that string. Calling the query here avoids repeating the interval comparison in several places.

Design principle

Keep one responsibility visible in each operation: the query evaluates the rule; the update changes the reading; the report constructs text; the caller displays it. These roles belong to this design, not to a Python restriction that forces every method with a particular name to behave that way.

2.7. Returned values are separate from printed output

Lecture-slide alignment: Week 8 source PDF, p. 40. PDF page numbers include title and divider pages.

The following example makes both the report's returned string and the update's returned `None` visible.

Slide example. Source PDF page(s) 40.

```
# ENG 101 Week 8: A result and a state change differ
# Basis: source PDF page(s) 40; slide example.
from sensor_models import Sensor  # The five-field interval Sensor.

s = Sensor("Tank temperature", "deg C", 82.5, 0.0, 90.0)
text = s.report()             # Return text; do not print inside report().
print(text)                   # Display the result once.
result = s.update(95.0)       # Change the reading; receive None.
print(result)                 # Show the method's returned value.
print(s.value)                # Inspect the independently changed state.
```

Code 2.4: A result and a state change differ

Example file: `ex13_return_and_print.py`

```
Tank temperature: 82.5 deg C | WITHIN LIMITS
None
95.0
```

The name `text` receives the result of `report()`. Calling `print(text)` then displays it once. The name `result` receives the result of `update(95.0)`, which is `None`. Meanwhile, reading `s.value` shows that the sensor's state has changed to `95.0`.

There is no contradiction between a method returning `None` and performing useful work. The contract must make the intended effect explicit. For this update method, inspect both the mutation and the return rather than assuming one tells you everything about the other.

2.8. Boundary cases and a complete update trace

Lecture-slide alignment: Week 8 source PDF, pp. 41–42. PDF page numbers include title and divider pages.

The source's interval is 0.0 to 90.0, inclusive. An ordinary interior reading is insufficient to check whether the comparison operators at the endpoints are correct. The test sequence deliberately includes values just outside, exactly on, and well inside the interval.

Slide example. Source PDF page(s) 41.

```python
# ENG 101 Week 8: Test both inclusive endpoints
# Basis: source PDF page(s) 41; slide example.
from sensor_models import Sensor  # This version assumes valid inputs.

s = Sensor("Temperature", "deg C", 50.0, 0.0, 90.0)
# Check outside, on, and inside the stated interval.
for value in [-0.1, 0.0, 50.0, 90.0, 90.1]:
    s.update(value)                      # Store this test reading.
    print(f"{s.value:5.1f}: {s.is_safe()}")  # Query the current reading.
```

Code 2.5: Test both inclusive endpoints

Example file: `ex14_interval_boundaries.py`

```
 -0.1: False
  0.0: True
 50.0: True
 90.0: True
 90.1: False
```

For `-0.1`, the first part of the chained comparison is false. For `0.0`, both comparisons are true. For `90.0`, equality at the upper boundary is included. For `90.1`, the upper comparison is false. In this first generic sensor model, a negative reading is not rejected merely for being negative; it is classified using the supplied interval. Do not confuse this with the later absolute-pressure contract.

A report trace shows how a changing reading produces a changing query result without changing the sensor's name, unit, or limits.

Slide example. Source PDF page(s) 42.

```python
# ENG 101 Week 8: Report before and after updating
# Basis: source PDF page(s) 42; slide example.
from sensor_models import Sensor  # Query/update/report model.

s = Sensor("Tank temperature", "deg C", 82.5, 0.0, 90.0)
print(s.report())            # Evaluate the initial current state.
s.update(95.0)               # Replace only this instance's reading.
print(s.report())            # Generate a new report from new state.
```

Code 2.6: Report before and after updating

Example file: `ex15_update_trace.py`

```
Tank temperature: 82.5 deg C | WITHIN LIMITS
Tank temperature: 95.0 deg C | OUTSIDE LIMITS
```

Point in execution	value	Query result	Report status
After construction	82.5	True	WITHIN LIMITS
After `update(95.0)`	95.0	False	OUTSIDE LIMITS

The changed result is obtained by calling the query again. It is not a message automatically pushed into every variable that previously received a report.

2.9. Four method contracts to recognize

Lecture-slide alignment: Week 8 source PDF, p. 43. PDF page numbers include title and divider pages.

Kind	Example	Changes state?	Returns
Calculation	`material.mass_from_volume(v)`	No	Number
Query	`sensor.is_safe()`	No	Boolean
Update	`sensor.update(x)`	Yes	`None` here
Report	`sensor.report()`	No	String here

The calculation example is developed in Section 2.12. A calculation combines stored information with arguments supplied for that call. A query answers a question about current state. A report presents that state as text. An update deliberately changes it.

For each operation, write a sentence saying what it reads, what it changes, and what its call returns. This habit makes the interface usable by someone who has not inspected the body. It also identifies what tests should check: a non-mutating report should leave the stored fields unchanged, while an update should change only the intended field.

2.10. Prediction activity: an old result or current state?

Lecture-slide alignment: Week 8 source PDF, p. 44. PDF page numbers include title and divider pages.

The next script stores a Boolean result before changing the sensor. Predict all three printed lines.

Slide example. Source PDF page(s) 44.

```
# ENG 101 Week 8: An old result is not a live query
# Basis: source PDF page(s) 44; slide example.
from sensor_models import Sensor  # Reuse the interval model.

s = Sensor("Test", "deg C", 82.5, 0.0, 90.0)
answer = s.is_safe()          # Compute True at this point in execution.
s.update(95.0)                # Change state after answer was calculated.
print(answer)                 # The previously stored bool stays True.
print(s.is_safe())            # A fresh query now returns False.
print(s.value)                # The current reading is 95.0.
```

Code 2.7: An old result is not a live query

Example file: `ex16_stored_boolean.py`

```
True
False
95.0
```

Answer and explanation

`answer` is calculated while the reading is 82.5, inside the interval, so it receives `True`. Updating the sensor to 95.0 does not re-evaluate that previous assignment. The old Boolean remains true; a new call to `s.is_safe()` returns false; the stored reading is 95.0.

The same principle applies to a previously stored report string. A result describes the state at the time the call ran. It is not a permanently live view of the sensor. When current state is required, make a new query or report call.

2.11. A method reference is not a decision result

Lecture-slide alignment: Week 8 source PDF, p. 45. PDF page numbers include title and divider pages.

The following program is deliberately wrong but does not raise an exception. The missing parentheses leave `check` referring to a bound method object instead of a Boolean.

Deliberately incorrect logic. Slide example. Source PDF page(s) 45.

```python
# ENG 101 Week 8: Deliberate missing-parentheses defect
# Basis: source PDF page(s) 45; slide example.
from sensor_models import Sensor  # The five-field Sensor class.

s = Sensor("Tank temperature", "deg C", 95.0, 0.0, 90.0)
check = s.is_safe             # BUG: method object, not a Boolean result.
if check:                     # A bound method object is truthy.
    print("WRONG branch: method was not called")
print(s.is_safe())            # The actual query result is False.
```

Code 2.8: Deliberate missing-parentheses defect

Example file: `ex17_forgotten_call.py`

```
WRONG branch: method was not called
False
```

A bound method object is truthy. Therefore `if check:` takes the branch even though executing the query would return false. Seeing a branch execute successfully is not evidence that the intended condition was evaluated.

Worked repair. Source PDF page(s) 45.

```python
# ENG 101 Week 8: Call the query before making a decision
# Basis: source PDF page(s) 45; worked repair.
from sensor_models import Sensor  # Reuse the same test conditions.

s = Sensor("Tank temperature", "deg C", 95.0, 0.0, 90.0)
check = s.is_safe()           # Repair: evaluate the method call.
if check:
    print("Within limits")
else:
    print("Outside limits")
assert check is False       # Keep a test that challenges the repair.
```

Code 2.9: Call the query before making a decision

Example file: `ex18_call_repair.py`

```
Outside limits
```

The repair changes `s.is_safe` to `s.is_safe()`. The subsequent assertion checks the specific false case that exposed the mistake. A test using only a reading inside the limits might fail to reveal the original defect because both the method object's truthiness and the actual query would lead to the same branch.

Common mistake

When a Boolean decision is unexpectedly true, inspect both the value and what was actually evaluated. A method, a nonempty string, or another truthy object can be accepted by `if` without being the Boolean answer you intended.

2.12. Calculate with a material's stored density

Lecture-slide alignment: Week 8 source PDF, pp. 46–47. PDF page numbers include title and divider pages.

The material class stores density as an attribute and receives a part volume as a method argument. These are different responsibilities: density belongs to the chosen material model, while each calculation may use a different part volume.

```python
class Material:
    """Store a name and an illustrative constant density in kg/m^3."""

    def __init__(self, name, density_kg_m3):
        # Assume positive finite density; this version does not check it.
        self.name = name
        self.density_kg_m3 = density_kg_m3

    def mass_from_volume(self, volume_m3):
        """Return kg; assume a finite nonnegative volume in m^3."""
        # Uniform constant-density model: mass = density * volume.
        # Do not store the part volume as a material property.
        return self.density_kg_m3 * volume_m3
```

Code 2.10: Annotated definition of `Material`

Extract from `examples/sensor_models.py`; use that complete file when running the examples.

The model is

$$m = \rho V, \qquad \left[\frac{\text{kg}}{\text{m}^3}\right]\left[\text{m}^3\right] = [\text{kg}].$$

The assumptions are constant, uniform density and compatible finite inputs. The class documents those assumptions but does not add type, finiteness, or positivity validation. The method returns a computed number and leaves the stored density unchanged.

Slide example. Source PDF page(s) 46-47.

```python
# ENG 101 Week 8: Calculate mass from stored density
# Basis: source PDF page(s) 46-47; slide example.
from sensor_models import Material  # Definition appears in these notes.

# Densities are slide practice data, not certified engineering values.
aluminum = Material("Aluminum", 2700.0)
steel = Material("Steel", 7850.0)
volume_m3 = 0.002                       # Same part volume for both calls.
print(f"Aluminum mass = {aluminum.mass_from_volume(volume_m3):.2f} kg")
print(f"Steel mass = {steel.mass_from_volume(volume_m3):.2f} kg")
# Independent check: 2700 * 0.002 = 5.4 kg under this model.
assert abs(aluminum.mass_from_volume(volume_m3) - 5.4) < 1e-12
```

Code 2.11: Calculate mass from stored density

Example file: `ex19_material_mass.py`

```
Aluminum mass = 5.40 kg
Steel mass = 15.70 kg
```

The expected aluminum result is independently calculated as $2700 \times 0.002 = 5.4$ kg. For the illustrative steel density, $7850 \times 0.002 = 15.7$ kg. The test uses an absolute tolerance of 10^{-12} kg for this small arithmetic example; that tolerance is not a claim about measurement accuracy or

material-property certainty.

Engineering check

The slide densities are programming practice data, not certified design properties. The calculation does not account for density variation, uncertainty, or a different unit convention. Returning two decimal places does not make those omitted physical assumptions more accurate.

2.13. Use objects in familiar lists and loops

Lecture-slide alignment: Week 8 source PDF, p. 48. PDF page numbers include title and divider pages.

A list can contain references to instances just as it can contain references to other values. No new loop syntax is required.

Slide example. Source PDF page(s) 48.

```python
# ENG 101 Week 8: Process a list of engineering instances
# Basis: source PDF page(s) 48; slide example.
from sensor_models import Sensor  # Each element supports report().

# Each list element references a separate Sensor with its own units.
sensors = [
    Sensor("Tank temperature", "deg C", 82.5, 0.0, 90.0),
    Sensor("Pipe pressure", "kPa", 315.0, 100.0, 400.0),
    Sensor("Battery voltage", "V", 10.8, 11.0, 12.8),
]
for sensor in sensors:
    print(sensor.report())   # The loop variable selects the receiver.
```

Code 2.12: Process a list of engineering instances

Example file: `ex20_sensor_list.py`

```
Tank temperature: 82.5 deg C | WITHIN LIMITS
Pipe pressure: 315.0 kPa | WITHIN LIMITS
Battery voltage: 10.8 V | OUTSIDE LIMITS
```

At the first iteration, the loop variable `sensor` refers to the temperature instance. At the second, it refers to the pressure instance. At the third, it refers to the battery-voltage instance. Each call to `sensor.report()` therefore uses that receiver's own attributes.

The temperatures, pressures, and voltages are not combined arithmetically. Each object evaluates a rule in its own unit. Adding all three numeric readings would not be a meaningful engineering summary simply because all three values are floats. The common report interface allows the loop to process heterogeneous measurement meanings without losing their labels and limits.

2.14. Filter whole entities rather than isolated numbers

Lecture-slide alignment: Week 8 source PDF, p. 49. PDF page numbers include title and divider pages.

Filtering can retain the complete sensor references that fail their stated interval checks. This preserves names, units, current readings, limits, and available methods.

Slide example. Source PDF page(s) 48-49.

```python
# ENG 101 Week 8: Retain complete flagged entities
# Basis: source PDF page(s) 48-49; slide example.
from sensor_models import Sensor  # Reuse the interval model.

# Repeat the preceding prerequisite list so this file runs independently.
sensors = [
    Sensor("Tank temperature", "deg C", 82.5, 0.0, 90.0),
    Sensor("Pipe pressure", "kPa", 315.0, 100.0, 400.0),
    Sensor("Battery voltage", "V", 10.8, 11.0, 12.8),
]
flagged = []                # Start with no selected references.
for sensor in sensors:
    if not sensor.is_safe():
        flagged.append(sensor)  # Keep the object, not only its number.
for sensor in flagged:
    print(sensor.name)      # Labels remain available after filtering.
```

Code 2.13: Retain complete flagged entities

Example file: `ex21_filter_objects.py`

```
Battery voltage
```

Only the battery reading of 10.8 V is outside its interval of 11.0–12.8 V in this dataset. The resulting `flagged` list contains a reference to that sensor, not a copy of its number and not a new sensor.

A consequence of the source's reference-based filter

The list records which references were selected when the loop ran. If a selected sensor is later updated, the object visible through the filtered list changes too. Membership is not automatically recomputed. This follows from the same aliasing behavior developed next; Example `ex44` in the practice material makes the consequence explicit.

2.15. Aliasing and deliberate snapshots

Lecture-slide alignment: Week 8 source PDF, pp. 50–51. PDF page numbers include title and divider pages.

The assignment `backup = original` does not call `Sensor`. It binds a second name to the same instance. Updating through either name therefore changes the single shared object.

Slide example. Source PDF page(s) 50.

```python
# ENG 101 Week 8: Two names can select the same instance
# Basis: source PDF page(s) 50; slide example.
from sensor_models import Sensor  # A mutable instance.

original = Sensor("Tank", "deg C", 82.5, 0.0, 90.0)
backup = original            # Another name; no new Sensor is created.
backup.update(95.0)          # Update the shared object through its alias.
print(original.value, backup.value)  # Both observe the same state.
print(original is backup)    # is compares object identity.
```

Code 2.14: Two names can select the same instance

Example file: `ex22_aliasing.py`

```
95.0 95.0
True
```

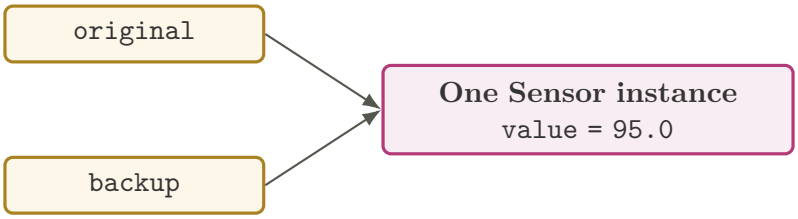


Figure 2.1.: Two names select one mutable object after `backup = original`. The diagram explains the source page 50 output.

`is` asks whether two references identify the same object. In this aliasing example it returns true. It is not a test that the two sensors happen to have equal numbers; there is only one sensor instance involved.

For the numeric and string fields in this class, an explicit new construction call can preserve a snapshot of the fields before a later update.

Slide example. Source PDF page(s) 51.

```python
# ENG 101 Week 8: Construct a separate snapshot
# Basis: source PDF page(s) 51; slide example.
from sensor_models import Sensor  # Here the fields are numbers/strings.

original = Sensor("Tank", "deg C", 82.5, 0.0, 90.0)
# A new instance is sufficient for these immutable field values.
snapshot = Sensor(
    original.name, original.unit, original.value,
    original.lower_limit, original.upper_limit,
)
original.update(95.0)                    # Change only the original.
print(original.value, snapshot.value)    # Snapshot retains 82.5.
print(original is snapshot)              # Different instances.
```

Code 2.15: Construct a separate snapshot

Example file: `ex23_snapshot.py`

```
95.0 82.5
False
```

The snapshot receives the field values currently stored in `original`, but it is a separate instance. Reassigning `original.value` therefore does not reassign `snapshot.value`. The output records 95.0 and 82.5, followed by false for the identity comparison.

This is sufficient for these immutable numeric and string fields. It is not a general promise that manually passing every attribute creates independent nested data. When an attribute refers to a mutable list, the copying policy must be deliberate. The source's optional reference covers that situation without introducing inheritance.

2.16. Debugging the receiver in the definition and call

Lecture-slide alignment: Week 8 source PDF, p. 52. PDF page numbers include title and divider pages.

Two complementary mistakes appear in the clinic. The first method definition has no parameter to accept the automatically supplied instance. The second definition has `self`, but the explicit class call supplies no receiver.

Intentional error; run separately. Slide example. Source PDF page(s) 52.

```
# ENG 101 Week 8: Intentional missing receiver parameter
# Basis: source PDF page(s) 52; slide example.
# Intentional defect: this ordinary instance method needs self.
class Sensor:
    def report():              # BUG: no parameter accepts the receiver.
        return "Reading"

s = Sensor()                   # Create a receiver.
s.report()                     # Python supplies s; TypeError follows.
```

Code 2.16: Intentional missing receiver parameter

Example file: `intentional_errors/ex24_missing_self.py`

```
TypeError: Sensor.report() takes 0 positional arguments but 1 was given
```

Only the final exception line is shown; traceback paths and version-dependent wording are omitted.

Intentional error; run separately. Slide example. Source PDF page(s) 52.

```
# ENG 101 Week 8: Intentional call without a receiver
# Basis: source PDF page(s) 52; slide example.
class Sensor:
    def report(self):
        return "Reading"       # This ordinary method requires self.

# Intentional defect: the class call does not automatically select an instance.
Sensor.report()                # No argument supplies self; TypeError.
```

Code 2.17: Intentional call without a receiver

Example file: `intentional_errors/ex25_no_receiver.py`

```
TypeError: Sensor.report() missing 1 required positional argument: 'self'
```

Only the final exception line is shown; traceback paths and version-dependent wording are omitted.

Both errors are argument-binding problems. In the first, Python supplies one receiver argument to a definition expecting none. In the second, an ordinary function stored on the class is called

without the required first argument. Exact error-message wording may vary with Python version, but the missing or surplus argument is the diagnostic fact.

Worked repair. Source PDF page(s) 52.

```python
# ENG 101 Week 8: Repair both receiver mistakes
# Basis: source PDF page(s) 52; worked repair.
# Worked repair: accept the receiver and call through an instance.
class Sensor:
    def report(self):           # self receives the particular instance.
        return "Reading"       # A simple string is returned.

s = Sensor()
assert s.report() == "Reading"  # Test the ordinary calling syntax.
print(s.report())
```

Code 2.18: Repair both receiver mistakes

Example file: `ex26_receiver_repair.py`

```
Reading
```

The worked repair supplies `self` in the method definition and calls through a specific instance. It does not remove the receiver from the design or turn the method into an unrelated global function.

2.17. Lecture 23 exit ticket and summary

Lecture-slide alignment: Week 8 source PDF, p. 53. PDF page numbers include title and divider pages.

1. Data, method, and call. `sensor.value` reads a stored number. `sensor.report` obtains the method without running it. `sensor.report()` executes it and returns text.

2. Update contract. In the current class, `sensor.update(95.0)` replaces only the receiver's current value and returns `None`. It does not create a new sensor or validate every possible input.

3. Aliasing. `backup = sensor` makes another name for the same instance. A later update through `backup` is visible through `sensor` because the object, not its name, holds the changed state.

4. Two useful tests. Set the reading exactly equal to `upper_limit` and expect a true query. Separately, create two instances, update one, and verify that the other's value remains unchanged. These test a boundary rule and the independence of distinct construction calls.

Section summary

A method acts on a selected instance. Its contract must make state changes and returned results clear. Read stored attributes, call behavior with parentheses, preserve meaningful metadata, and distinguish an alias from a new instance.

3 Reliable Engineering Objects

Lecture-slide alignment: Week 8 source PDF, pp. 54–78. PDF page numbers include title and divider pages.

3.1. Learning goals: state has a contract too

Lecture-slide alignment: Week 8 source PDF, pp. 54–55. PDF page numbers include title and divider pages.

Lecture 24 connects the new class syntax with Week 7's validation and testing. The central idea is simple: if an object stores a current engineering reading, a failed update should not quietly corrupt that reading.

You will write a state contract before adding checks; distinguish invalid data from a valid reading outside an operating limit; validate a proposal before changing state; test initialization, boundaries, failures, and separate instances; combine ordinary functions with objects and loops; and plan a small class-based task with another student.

The example is an independent `PressureSensor` class. It is not built by inheriting from the generic `Sensor`. Its narrower meaning makes validation explicit: this class stores *absolute pressure in kPa*, a label, and an inclusive warning threshold.

3.2. Warm-up: invalid data or an exceeded limit?

Lecture-slide alignment: Week 8 source PDF, p. 56. PDF page numbers include title and divider pages.

The source warm-up uses a 400 kPa limit and four proposed readings. These distinguish whether a number belongs to the model's valid input domain from whether it meets the operating criterion.

Absolute pressure	Limit	Expected treatment
315 kPa	400 kPa	Store it; report within the limit.
400 kPa	400 kPa	Store it; equality is included.
900 kPa	400 kPa	Store it; report that the limit is exceeded.
−5 kPa	400 kPa	Reject under this absolute-pressure contract.

A reading of 900 kPa is above the selected threshold, but it is still valid data under the stated teaching contract. The object must retain it and report the exceedance. Rejecting every high reading as malformed input would remove the very evidence that the monitoring program is intended to reveal.

The negative proposal is different: it violates the nonnegative domain explicitly selected for this absolute-pressure model. This rule must not be applied indiscriminately to other quantities or to a gauge-pressure interpretation. The important course lesson is to specify the quantity before deciding its domain.

Warm-up explanation

Silently discarding a high but valid reading would leave an older, lower reading visible and could misrepresent the latest accepted state. Acceptable input and acceptable operating condition are different classifications. Preserve the valid high value and make the exceeded criterion visible.

3.3. Define one responsibility and its limits

Lecture-slide alignment: Week 8 source PDF, p. 57. PDF page numbers include title and divider pages.

One `PressureSensor` instance stores its label, current absolute pressure, and inclusive warning threshold. It accepts or rejects proposed readings according to the input contract, then returns reports from the state actually stored.

It does not read physical hardware, control a valve, stop a process, certify a system, or represent every possible sensor type. Those are deliberately excluded responsibilities. Keeping them outside this class makes it possible to inspect the full state-changing behavior in a small example.

The ordinary caller is responsible for supplying proposed readings and deciding what to do with returned text or exceptions. This separation is useful even when all parts of the program run in one Python script. The input source, object model, and display operation need not be the same component.

3.4. Write the input and stored-state contract

Lecture-slide alignment: Week 8 source PDF, p. 58. PDF page numbers include title and divider pages.

Item	Teaching contract in the source
Quantity and unit	Absolute pressure in kPa; one fixed unit convention.
Numeric inputs	Integers or floats convertible to a finite float; Boolean values excluded.
Domain	The pressure and selected threshold are nonnegative.
Operating criterion	`value_kpa <= limit_kpa`; equality is included.
Failed update	Raise the appropriate exception and preserve the previous reading.

The stored-state expectation applies after successful initialization and after a successful update through the documented interface. The numeric fields are standardized to floats, and the unit field is fixed to `"kPa (absolute)"`. A name must be nonempty text after surrounding whitespace is removed.

The word *finite* matters. NaN and infinities are floating-point values, but they do not satisfy this model's finite-value contract. Boolean values also need separate attention because Python recognizes `bool` as related to `int`; the helper explicitly excludes them rather than treating true as a pressure of one.

The intended domain and the exact implementation

Source pages 58 and 79 describe built-in `int`/`float` inputs. The supplied helper actually uses `isinstance`, which also recognizes subclasses of those types. The helper is preserved, not silently rewritten. For the ordinary inputs used this week the behavior matches the intended policy; the precise qualification is recorded in Section 4.1. No inheritance syntax is needed for the lecture examples.

3.5. Validate before committing a change

Lecture-slide alignment: Week 8 source PDF, p. 59. PDF page numbers include title and divider pages.

A checked update is best traced as two phases: validate a proposed value, then assign it. If validation raises an exception, execution must leave the method before reaching the state-changing assignment.

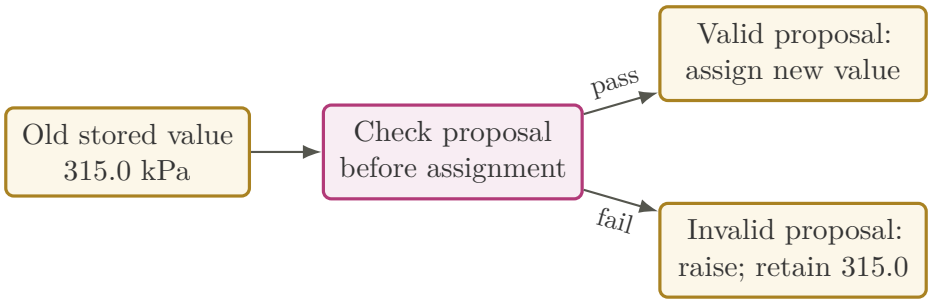


Figure 3.1.: The failed-update path does not reach the assignment. Adapted from source PDF page 59.

The failure guarantee is not achieved merely by writing an `except` block in the caller. It depends on the ordering of statements inside the method. If the method first stored an invalid reading and only then raised an error, catching the exception would not undo the state change.

In this class the only committed field in `update` is `self.value_kpa`. The helper returns a valid candidate first, then the assignment stores it. This is a small, inspectable example of separating a proposed change from an accepted change.

3.6. An ordinary function can support a class

Lecture-slide alignment: Week 8 source PDF, p. 60. PDF page numbers include title and divider pages.

The reusable helper is `checked_nonnegative(value, label)`. It returns a validated float; rejects unsuitable types with `TypeError`; and rejects negative, nonfinite, or excessively large values with `ValueError`. The label is used in the error message so that a caller can distinguish a problem in `limit_kpa` from a problem in `new_value_kpa`.

The helper belongs outside the class because the same check is useful for the initial reading, the threshold, and later update proposals. This is not a competition between functions and classes. The function performs a stateless validation task; the object stores the accepted state and provides related operations.

The complete annotated helper is printed in Section 4.1, following the source's optional-reference placement. The working file `pressure_sensor.py` contains the required `import math`, then the helper, then the class, so it runs without missing definitions.

3.7. The checked pressure class, assembled

Lecture-slide alignment: Week 8 source PDF, pp. 61–63. PDF page numbers include title and divider pages.

The next listing combines the initializer, update, query, and report shown on the three source pages. It retains their order and behavior. The numeric helper named in this code is the function just specified.

```python
class PressureSensor:
    """Store absolute pressure in kPa and an inclusive warning limit.

    High valid readings are retained. Rejected updates preserve state.
    Public attributes remain writable: use the documented update route.
    """

    def __init__(self, name, value_kpa, limit_kpa):
        # The slide uses ValueError for both wrong-type and empty names.
        if not isinstance(name, str) or not name.strip():
            raise ValueError("name must be nonempty text")
        self.name = name.strip()           # Remove surrounding spaces.
        self.unit = "kPa (absolute)"       # Fix the unit convention.
        # Validate each numerical field during initialization.
        self.limit_kpa = checked_nonnegative(limit_kpa, "limit_kpa")
        self.value_kpa = checked_nonnegative(value_kpa, "value_kpa")

    def update(self, new_value_kpa):
        """Store a valid reading; on failure leave the old state unchanged."""
        # Complete validation before the assignment can change state.
        checked = checked_nonnegative(new_value_kpa, "new_value_kpa")
        self.value_kpa = checked           # Commit only a valid value.
        # The successful call returns None, not a replacement sensor.

    def is_safe(self):
        """Return whether pressure is no greater than the chosen limit."""
        # Equality is included. This query does not operate hardware.
        return self.value_kpa <= self.limit_kpa

    def report(self):
        """Return reading, criterion, and result without changing state."""
        if self.is_safe():                 # Evaluate the current state.
            status = "WITHIN LIMIT"
        else:
            status = "LIMIT EXCEEDED"
        # A high reading stays visible alongside the chosen threshold.
        return (f"{self.name}: {self.value_kpa:.1f} {self.unit} | "
                f"limit={self.limit_kpa:.1f} kPa | {status}")
```

Code 3.1: Annotated definition of `PressureSensor`

Extract from `examples/pressure_sensor.py`; use that complete file when running the examples.

3.7.1. Trace the initializer line by line

The name condition uses short-circuit `or`. If `name` is not a string, the condition is already true and Python does not attempt `name.strip()`. If it is a string, `strip()` removes surrounding whitespace and the empty result is treated as false, so `not name.strip()` detects blank text. The source raises `ValueError` for both a non-string name and an empty name; these notes retain

that exact behavior.

After the name check, the initializer stores the cleaned name and fixed unit label. It validates the threshold and reading through the helper. A failed numeric validation prevents successful completion of this construction call. The code does not pretend that creating a Python record establishes a connection to a physical sensor.

3.7.2. Trace the update separately

The update receives a proposed value as `new_value_kpa`. The helper call is evaluated before its result is assigned to the local variable `checked`. If an exception is raised, neither the local assignment nor the later attribute assignment completes. If the helper returns, `self.value_kpa = checked` stores the validated number. The successful method returns `None`.

3.7.3. Read the query's assumption

The query checks only `self.value_kpa <= self.limit_kpa`. It relies on initialization and documented updates to maintain valid nonnegative stored numbers. It does not revalidate every attribute on every call. That distinction becomes important when considering direct public-attribute assignment in Section 3.14.

3.7.4. Read the report's evidence

The report evaluates the current query and selects `WITHIN LIMIT` or `LIMIT EXCEEDED`. Unlike the generic interval report, it prints the numeric threshold as well as the reading and unit. The returned string is evidence of why the current result was selected, not a command that operates hardware.

3.8. A high reading remains visible

Lecture-slide alignment: Week 8 source PDF, p. 64. PDF page numbers include title and divider pages.

This example begins below the selected limit and then updates to a valid reading above it.

Slide example. Source PDF page(s) 64.

```
# ENG 101 Week 8: Retain a valid high pressure
# Basis: source PDF page(s) 64; slide example.
from pressure_sensor import PressureSensor  # Checked absolute-pressure model.

p = PressureSensor("Pipe", 315.0, 400.0)     # All pressures are in kPa.
print(p.report())                            # Initial value is below limit.
p.update(900.0)                              # Valid input, outside limit.
print(p.report())                            # Show the high reading, not zero.
```

Code 3.2: Retain a valid high pressure

Example file: `ex27_pressure_high.py`

```
Pipe: 315.0 kPa (absolute) | limit=400.0 kPa | WITHIN LIMIT
Pipe: 900.0 kPa (absolute) | limit=400.0 kPa | LIMIT EXCEEDED
```

The second report contains 900.0, not the old 315.0 and not a substituted zero. The high

value satisfies the numeric input contract even though its query result is false. The label `LIMIT EXCEEDED` makes that situation explicit.

Engineering check

A monitoring program should not make an inconvenient observation disappear by treating every exceeded threshold as an input-conversion failure. Distinguish "the reading was rejected" from "the reading was accepted and violates the operating rule."

3.9. Test a normal value, the boundary, and retained data

Lecture-slide alignment: Week 8 source PDF, p. 65. PDF page numbers include title and divider pages.

The tests begin at zero, move to the exact threshold, then move just above it. Each input has an expectation derived from the written contract.

Slide example. Source PDF page(s) 65.

```python
# ENG 101 Week 8: Check state as well as classification
# Basis: source PDF page(s) 65; slide example.
from pressure_sensor import PressureSensor  # Values must be nonnegative.

p = PressureSensor("Test", 0.0, 400.0)       # Zero is valid in this model.
assert p.is_safe()
p.update(400.0)
assert p.is_safe()                           # Exact boundary is included.
p.update(400.1)
assert not p.is_safe()                       # Query detects exceedance.
assert p.value_kpa == 400.1                  # The high value was retained.
print("Pressure value and boundary checks passed")
```

Code 3.3: Check state as well as classification

Example file: `ex28_pressure_boundaries.py`

```
Pressure value and boundary checks passed
```

Zero is accepted under this mathematical model. A stored value of 400.0 satisfies an inclusive limit of 400.0. The value 400.1 must be stored and produce a false query. The last assertion checks that the high reading was actually retained; otherwise, a defective update that discarded the proposal could hide the exceedance.

These assertions use directly assigned values. A separate numerical tolerance would be needed when checking the result of an approximate computation. Here the principal evidence is correct branching and state retention, not numerical approximation error.

3.10. A failed update needs two checks

Lecture-slide alignment: Week 8 source PDF, pp. 66–67. PDF page numbers include title and divider pages.

The first demonstration preserves the source page 66 structure: save the old value, attempt an invalid update, catch the expected exception, and inspect the remaining value.

Slide example. Source PDF page(s) 66.

```python
# ENG 101 Week 8: Observe a rejected update
# Basis: source PDF page(s) 66; slide example.
from pressure_sensor import PressureSensor  # Uses explicit validation.

p = PressureSensor("Pipe", 315.0, 400.0)
before = p.value_kpa                        # Save the previous reading.
try:
    p.update(-5.0)                          # Invalid absolute pressure.
except ValueError:
    print("Rejected invalid absolute pressure")
# This assertion tests preservation; the next example also requires rejection.
assert p.value_kpa == before
print(f"Stored reading = {p.value_kpa:.1f} kPa")
```

Code 3.4: Observe a rejected update

Example file: `ex29_failed_update_demo.py`

```
Rejected invalid absolute pressure
Stored reading = 315.0 kPa
```

The observed message shows that a `ValueError` was raised on this run. The assertion then shows that the old reading remains. Source page 67 asks you to strengthen this into a complete repeatable test.

Checking unchanged state alone is not enough

A broken method that silently ignores every update would leave the old reading unchanged too. Therefore a test must establish *both* that the required exception occurred and that no unintended change was committed.

The worked completion below adds an `else` clause to the `try/except`. That clause runs when the attempt does not raise the expected exception, so it makes silent non-rejection fail the test.

Worked completion of the failure-test activity. Source PDF page(s) 67, 78.

```python
# ENG 101 Week 8: Require an exception and preserved state
# Basis: source PDF page(s) 67, 78; worked completion of the failure-test activity.
from pressure_sensor import PressureSensor  # The same checked class.

p = PressureSensor("Pipe", 315.0, 400.0)
before = p.value_kpa                        # State before the experiment.
try:
    p.update(-5.0)                          # The contract requires rejection.
except ValueError:
    pass                                    # Expected exception was observed.
else:
    raise AssertionError("Expected ValueError was not raised")
# A silently ignored input fails the else branch; a corrupting update fails here.
assert p.value_kpa == before
print("Expected rejection and state preservation confirmed")
```

Code 3.5: Require an exception and preserved state

Example file: `ex30_expected_failure_test.py`

```
Expected rejection and state preservation confirmed
```

The `pass` statement does not hide an unknown error here: it is inside a handler for the specific exception required by the test. A different exception is not swallowed. The explicit `raise AssertionError` in the `else` branch tests the absence of the expected failure; the final assertion tests preserved state.

This is a good example of why an exception test is an experiment. State the required failure first, choose an input that should trigger it, and verify what did and did not change.

3.11. Check independence with a second object

Lecture-slide alignment: Week 8 source PDF, p. 68. PDF page numbers include title and divider pages.

A single object cannot expose every accidental-sharing defect. The source therefore constructs two pressure sensors and updates only the first.

Slide example. Source PDF page(s) 68.

```python
# ENG 101 Week 8: A second instance exposes shared-state errors
# Basis: source PDF page(s) 68; slide example.
from pressure_sensor import PressureSensor  # Create two independent records.

first = PressureSensor("A", 100.0, 400.0)
second = PressureSensor("B", 100.0, 400.0)
first.update(450.0)                          # Update only the selected instance.
assert first.value_kpa == 450.0
assert second.value_kpa == 100.0            # It must remain unchanged.
assert first is not second                  # Verify distinct identities.
print("Independent-instance checks passed")
```

Code 3.6: A second instance exposes shared-state errors

Example file: `ex31_pressure_independence.py`

```
Independent-instance checks passed
```

The first two assertions inspect observable state after the operation. The third checks identity directly. Equal initial readings do not imply a single shared sensor: the two construction calls produce different instances. Conversely, giving a second name to one instance would not produce independence.

A useful failure report for this test would show both values, not merely a generic message that a check failed. The independent expected value for the second object is still 100.0 because no update was requested for that receiver.

3.12. Combine objects, loops, and exception handling

Lecture-slide alignment: Week 8 source PDF, pp. 69–70. PDF page numbers include title and divider pages.

The monitoring loop uses fixed proposals so that its behavior can be traced independently. The caller attempts one update per loop pass. It reports rejected data explicitly; only a successful update reaches the `else` branch that prints a new reading report.

Slide example. Source PDF page(s) 69-70.

```python
# ENG 101 Week 8: A reproducible pressure-monitoring sequence
# Basis: source PDF page(s) 69-70; slide example.
from pressure_sensor import PressureSensor  # Validation stays in the model.

p = PressureSensor("Pipe", 315.0, 400.0)
# Fixed proposals allow an independent state trace, unlike random input.
for proposed_kpa in [315.0, 400.0, 900.0, -5.0, 380.0]:
    try:
        p.update(proposed_kpa)              # Attempt one checked update.
    except (TypeError, ValueError) as error:
        print(f"REJECTED {proposed_kpa}: {error}")
    else:
        print(p.report())                   # Report an accepted new reading only.
```

Code 3.7: A reproducible pressure-monitoring sequence

Example file: `ex32_monitoring_loop.py`

```
Pipe: 315.0 kPa (absolute) | limit=400.0 kPa | WITHIN LIMIT
Pipe: 400.0 kPa (absolute) | limit=400.0 kPa | WITHIN LIMIT
Pipe: 900.0 kPa (absolute) | limit=400.0 kPa | LIMIT EXCEEDED
REJECTED -5.0: new_value_kpa must be finite and nonnegative
Pipe: 380.0 kPa (absolute) | limit=400.0 kPa | WITHIN LIMIT
```

Proposed kPa	Accepted?	Stored afterward	Event reported
315.0	Yes	315.0	Within limit
400.0	Yes	400.0	Boundary included
900.0	Yes	900.0	Limit exceeded
−5.0	No	900.0	Proposal rejected; old value retained
380.0	Yes	380.0	Within limit

Notice that rejecting −5.0 in this sequence retains *900.0*, not 315.0. The unchanged state is the value immediately before the rejected update, not always the initial value. The separate demonstration in Section 3.10 retained 315.0 because no preceding high update had occurred there.

The `else` is important: printing a normal report after the failed update could be mistaken for a report about the rejected proposal. The source instead identifies the rejection separately. The old value is still available, but it is not a new accepted measurement for that loop pass.

3.13. Build an object-oriented test matrix

Lecture-slide alignment: Week 8 source PDF, p. 71. PDF page numbers include title and divider pages.

The same test-design discipline from Week 7 applies to methods. An object's persistent state simply adds another observation to record.

Case	Expected behavior	Evidence
Valid initialization	Expected fields established.	Name, unit, reading, threshold.
Exact operating limit	Query returns true.	Value and equality-boundary result.
Above the limit	Value stored; query false.	Reading retained, exceeded-limit report.
Invalid proposal	Exception; previous state retained.	Exception category and before/after state.
Two objects	Only selected instance changes.	Values in both objects and distinct identities.

A useful record has columns for the initial state, operation and arguments, expected return or exception, expected final state, and observed outcome. Recording only a final string can miss a wrong internal state that becomes visible on the next call. Recording only state can miss a wrong return value or a missing required exception.

Initialization and updates should both be tested. A class may validate incoming updates but incorrectly accept invalid initial data, or the reverse. The helper's type and finiteness cases in Chapter 4 provide further inputs with which to challenge that contract.

3.14. Public attributes can bypass the checked route

Lecture-slide alignment: Week 8 source PDF, p. 72. PDF page numbers include title and divider pages.

The class is deliberately plain Python. Its attributes are publicly writable. The call `p.update(-5.0)` is rejected, but `p.value_kpa = -5.0` does not automatically call `update`.

Deliberate misuse / design-limit demonstration. Worked illustration of the stated design limitation. Source PDF page(s) 72.

```python
# ENG 101 Week 8: A direct assignment bypasses the update method
# Basis: source PDF page(s) 72; worked illustration of the stated design
    limitation.
from pressure_sensor import PressureSensor  # Public fields are still writable.

p = PressureSensor("Pipe", 315.0, 400.0)
p.value_kpa = -5.0              # Deliberate misuse: skips update validation.
print(p.value_kpa)              # The invalid value has been stored.
print(p.is_safe())              # True: the query assumes valid stored state.
# This is a limitation demonstration, not recommended application code.
```

Code 3.8: A direct assignment bypasses the update method

Example file: `ex33_public_bypass.py`

```
-5.0
True
```

This labelled misuse demonstrates the boundary of the design. A direct assignment places an invalid number in the object. The query then evaluates $-5 \leq 400$ and returns true because it assumes the stored state has already been validated. Its true result is not evidence that the invalid absolute-pressure record satisfies the full contract.

Design principle

Use the documented update interface. A validation method does not, by itself, make all attributes protected. The source explicitly leaves controlled attribute access, getters/setters, and further information hiding for Week 9.

The notes do not silently redesign this class with advanced access-control mechanisms. Understanding the limitation is part of understanding the actual code supplied in the slides.

3.15. Choose a class only when it improves the design

Lecture-slide alignment: Week 8 source PDF, pp. 73–74. PDF page numbers include title and divider pages.

A class is a good candidate when one recognizable entity owns persistent state, several operations depend on that state, many instances share a common interface, and the design becomes easier to explain and test. A function may be enough for a single formula, a short conversion, or a task with no persistent entity.

The pair activity offers an ideal resistor, a material record, or a temperature sensor. Start by listing attributes with units. Propose one calculation or query. Add an update only when it has a genuine role; not every class needs mutable state. Specify a known-answer test and one failure case, then exchange interfaces with another pair.

One possible pair-activity answer: ideal resistor

The entity is one ideal linear resistor. Its `resistance_ohm` attribute records resistance. Its `current_a(voltage_v)` method evaluates $I = V/R$. A 4-ohm resistor at 12 V should return 3 A. Zero resistance should be rejected by the source's short checked initializer. No update is required for this particular task. The optional worked code assumes finite numeric inputs rather than promising complete type validation.

Review a partner's interface before reading the implementation. Can you state which unit to provide, which value is returned, which state might change, and how the operation fails? Unclear answers indicate an interface problem even before code execution.

3.16. Assignment 4 preparation and Laboratory 3 practice

Lecture-slide alignment: Week 8 source PDF, pp. 75–76. PDF page numbers include title and divider pages.

Assignment 4 preparation focuses on identifying the entity and state, specifying changes and returned values, and preparing known-answer and boundary tests. Before submitting assessed work, read the actual task and marking criteria, check collaboration and permitted-tool rules, and document assistance as required. These notes do not establish new assessment requirements.

The Laboratory 3 slide provides a separate, explicitly illustrative workflow: one interval sensor, an inclusive temperature range of 20–30 °C, and the readings 19.5, 20.0, 25.0, 30.0, and 30.5. The program should update and report each reading and count those outside the range.

Worked solution to the illustrative lab workflow. Source PDF page(s) 76.

```python
# ENG 101 Week 8: Laboratory 3 illustrative temperature workflow
# Basis: source PDF page(s) 76; worked solution to the illustrative lab workflow.
from sensor_models import Sensor  # Use the interval Sensor, not PressureSensor.

# A valid initial reading is chosen only to create the practice object.
sensor = Sensor("Lab temperature", "deg C", 25.0, 20.0, 30.0)
warning_count = 0                          # Count readings, not unique objects.
for reading in [19.5, 20.0, 25.0, 30.0, 30.5]:
    sensor.update(reading)                 # Reuse one instance for all readings.
    print(sensor.report())                 # Generate a fresh report each time.
    if not sensor.is_safe():
        warning_count += 1                 # Count exactly one warning per failure.
print(f"Warnings = {warning_count}")
print(f"Final stored value = {sensor.value:.1f} deg C")
assert warning_count == 2                  # Independent slide expectations.
assert sensor.value == 30.5
```

Code 3.9: Laboratory 3 illustrative temperature workflow

Example file: `ex34_lab3_practice.py`

```
Lab temperature: 19.5 deg C | OUTSIDE LIMITS
Lab temperature: 20.0 deg C | WITHIN LIMITS
Lab temperature: 25.0 deg C | WITHIN LIMITS
Lab temperature: 30.0 deg C | WITHIN LIMITS
Lab temperature: 30.5 deg C | OUTSIDE LIMITS
Warnings = 2
Final stored value = 30.5 deg C
```

The chosen initial value of 25.0 is only a setup value for this worked completion; the slide does not require that initialization value. The loop then processes exactly the five supplied readings. The first and last readings are outside the interval; both exact boundaries are inside. The expected warning count is therefore two, and the final stored reading is 30.5.

Reading (°C)	Within limits?	Warnings accumulated
19.5	No	1
20.0	Yes	1
25.0	Yes	1
30.0	Yes	1
30.5	No	2

The counter counts out-of-range *readings*, not distinct sensor instances. There is one sensor object and five update attempts. The object stores only the current reading; it does not automatically maintain the whole history. The raw list in the caller remains the record of the proposed sequence.

3.17. Review the design and answer the exit ticket

Lecture-slide alignment: Week 8 source PDF, pp. 77–78. PDF page numbers include title and divider pages.

The source review asks whether the class has one clear purpose; whether names, units, and responsibilities are explicit; whether queries use the current receiver rather than hidden global data; whether failed updates preserve state; whether tests cover boundaries, wrong inputs, and another instance; and whether reports show the reading and criterion.

The generic `Sensor.report()` displays the reading and status but not the numeric interval. The pressure report explicitly includes its numeric threshold. This is a useful distinction when applying the review checklist; the notes preserve both source implementations rather than claiming their reports contain identical information.

Exit-ticket answer. The proposal −5.0 violates the selected nonnegative absolute-pressure domain. The operation should raise `ValueError`, and the previous reading of 315.0 kPa should remain. Example `ex30` tests that the exception really occurs and that the stored value is unchanged. A test of unchanged state alone is insufficient because silent non-rejection would also preserve the old reading.

Lecture 24 summary

Specify the valid input domain separately from the operating rule. Validate a proposed change before committing it. Retain valid high readings, distinguish rejected proposals from accepted measurements, and test the exception together with the state left behind. Classes and ordinary functions can cooperate without requiring inheritance.

4 Optional Reference and Debugging Notes

Lecture-slide alignment: Week 8 source PDF, pp. 79–83. PDF page numbers include title and divider pages.

These sections retain the deck's optional-reference status. They make the displayed helper, list-ownership examples, resistor answer, and debugging table available in full. They do not introduce the inheritance or class-variable topics reserved for Week 9.

4.1. The complete checked-number helper

Lecture-slide alignment: Week 8 source PDF, p. 79; interface on p. 60. PDF page numbers include title and divider pages.

The helper below is the source function with explanatory comments and its required `math` import. It returns a number only after the type, conversion, finiteness, and domain checks have succeeded.

```python
# Week 8, source PDF pages 60-63 and 79, assembled in executable order.
# This model stores absolute pressure in kPa, not gauge pressure.
import math  # Required for the finiteness test in the slide helper.


def checked_nonnegative(value, label):
    """Return a finite nonnegative float under the slide's numeric policy.

    Reject bool, numeric strings, nonnumeric inputs, negative values,
    NaN, infinities, and integers too large to convert to float.
    The isinstance checks also recognize int/float subclasses; see notes.
    """
    # Check the type before converting; True is not a pressure reading.
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError(f"{label} must be an int or float, not bool")
    try:
        # Standardize accepted numeric input to a floating-point value.
        number = float(value)
    except OverflowError as error:
        # Translate conversion overflow into a meaningful model error.
        # 'from error' preserves the original exception as the cause.
        raise ValueError(f"{label} is too large for this model") from error
    # Negative values and nonfinite values violate this model's domain.
    if not math.isfinite(number) or number < 0:
        raise ValueError(f"{label} must be finite and nonnegative")
    return number  # Return the checked number; no object is changed.
```

Code 4.1: The source numeric helper with its required import

4.1.1. First check: is this an intended numeric input?

The first condition rejects a Boolean or an object that is not recognized as an integer or float by `isinstance`. A numeric string such as `"315"` is therefore rejected even though `float("315")`

would work in a separate conversion step. This is an intentional interface boundary: converting user-entered text and storing a checked numeric reading are separate operations.

Boolean values require an explicit check. `True` and `False` are not intended pressure readings, even though Python's type relationships allow them to participate in integer operations. Rejecting them before conversion prevents an accidental logical flag from becoming a measurement of 1.0 or 0.0.

4.1.2. Second check: can the value be represented as a float?

The `try` block converts an accepted numeric input to `float`. An extremely large integer can be an integer in Python while being too large for this floating-point model. The source catches `OverflowError` and raises a model-facing `ValueError`. The clause `from error` retains the original exception as the cause of the replacement exception.

This standardization is a deliberate representation choice. It does not mean that a float can represent every real value exactly or preserve every digit of an arbitrarily large integer. The contract requires a finite floating-point representation, not exact storage of all mathematical numbers.

4.1.3. Third check: is the converted value in the domain?

`math.isfinite(number)` is false for NaN and positive or negative infinity. It is true for zero. The second part, `number < 0`, rejects negative finite values. Both checks are needed: a numerical type alone is not evidence that a value satisfies the engineering domain. The documented behavior of `math.isfinite` is recorded in [8].

The function returns `number` only after the checks finish. It has not changed a sensor. That absence of mutation is exactly why the caller can validate before assigning a stored attribute.

Input	Expected outcome	Reason
`0`	`0.0`	Valid boundary and standardized type.
`315.0`	`315.0`	Finite and nonnegative.
`True`	`TypeError`	Boolean input excluded.
`"315"`	`TypeError`	Text conversion is outside this interface.
`-5.0`	`ValueError`	Negative value violates the domain.
`float("nan")`	`ValueError`	Nonfinite value.
`float("inf")`	`ValueError`	Infinite value.
`10**400`	`ValueError`	Integer-to-float conversion overflows.

Worked contract-test extension. Source PDF page(s) 60, 79.

```python
# ENG 101 Week 8: Explore the helper contract
# Basis: source PDF page(s) 60, 79; worked contract-test extension.
from pressure_sensor import checked_nonnegative  # Standalone numeric helper.

# Accepted examples standardize int/float input to float.
print(checked_nonnegative(0, "reading"))
print(checked_nonnegative(315.0, "reading"))
# These rejected cases target type, physical-domain, and finiteness checks.
for value in [True, "315", -5.0, float("nan"), float("inf")]:
    try:
        checked_nonnegative(value, "reading")
    except (TypeError, ValueError) as error:
        print(type(error).__name__)       # Display a stable exception category.
    else:
        raise AssertionError("Expected invalid input to be rejected")
```

Code 4.2: Explore the helper contract

Example file: `ex35_numeric_helper_tests.py`

```
0.0
315.0
TypeError
TypeError
ValueError
ValueError
ValueError
```

Precisely what the source type check enforces

The source prose says built-in `int`/`float` inputs only, but `isinstance(value, (int, float))` also accepts instances of subclasses of those types. This is documented Python behavior [7]. The helper has not been replaced with an exact-type check in these notes. The distinction is recorded so that the intended domain is not overstated; subclass design itself remains outside Week 8.

4.2. Each instance can own a fresh list

Lecture-slide alignment: Week 8 source PDF, p. 80. PDF page numbers include title and divider pages.

A list may be an attribute. The important question is where the list object is created and which instances refer to it. In the source `ReadingLog`, the empty-list expression runs inside every initializer call.

Slide class with worked demonstration calls. Source PDF page(s) 80.

```python
# ENG 101 Week 8: Each initialization creates a fresh list
# Basis: source PDF page(s) 80; slide class with worked demonstration calls.
class ReadingLog:
    def __init__(self, name):
        self.name = name            # Label this particular log.
        self.readings = []          # Evaluate a new list literal on every call.

    def add(self, value):
        # Teaching version assumes a valid numeric reading.
        self.readings.append(value)

# Calls added to demonstrate the slide's two displayed lists.
first = ReadingLog("A")
second = ReadingLog("B")
first.add(82.5)                     # Add only to the first log.
print(first.readings)
print(second.readings)              # The other list is still empty.
```

Code 4.3: Each initialization creates a fresh list

Example file: `ex36_fresh_lists.py`

```
[82.5]
[]
```

`first` and `second` are distinct instances. Each call evaluates `[]` separately, creating a separate list. The method `first.add(82.5)` appends to `first.readings`. It does not touch `second.readings`, which remains empty.

The demonstration calls are a worked completion of the source class and its displayed two-list output. The `add` method assumes a valid numeric reading and does not provide additional type validation. The lesson is ownership of separate mutable lists, not a complete measurement-storage system.

4.3. A supplied list may still be shared

Lecture-slide alignment: Week 8 source PDF, p. 81. PDF page numbers include title and divider pages.

Creating a new `ReadingLog` does not guarantee that every object referenced by its attributes is new. If the initializer assigns an existing list directly to `self.readings`, both the caller's name and the attribute refer to the same list.

Slide constructor with worked observation. Source PDF page(s) 81.

```python
# ENG 101 Week 8: Receiving a list is not copying it
# Basis: source PDF page(s) 81; slide constructor with worked observation.
class ReadingLog:
    def __init__(self, readings):
        self.readings = readings   # Alias the caller's existing list.

# Worked observations of the exact aliasing constructor.
raw = [82.5, 85.0]
log = ReadingLog(raw)
log.readings.append(90.0)          # The caller can observe this mutation.
print(raw)
print(log.readings is raw)        # The attribute and raw select one list.
```

Code 4.4: Receiving a list is not copying it

Example file: `ex37_input_list_alias.py`

```
[82.5, 85.0, 90.0]
True
```

The append is visible through `raw` because the list itself is shared. The new log instance is separate from the caller, but its *list attribute* refers to the caller's existing mutable object. Independence of the enclosing instance is not the same as independence of everything it references.

The second source constructor copies the supplied outer list.

Slide constructor with worked observation. Source PDF page(s) 81.

```python
# ENG 101 Week 8: Copy the supplied outer list
# Basis: source PDF page(s) 81; slide constructor with worked observation.
class ReadingLog:
    def __init__(self, readings):
        self.readings = readings.copy()  # Copy outer list, not nested objects.

# With immutable numeric elements this protects the caller's list.
raw = [82.5, 85.0]
log = ReadingLog(raw)
log.readings.append(90.0)          # Changes the new outer list only.
print(raw)
print(log.readings)
print(log.readings is raw)        # The two lists have different identities.
```

Code 4.5: Copy the supplied outer list

Example file: `ex38_input_list_copy.py`

```
[82.5, 85.0]
[82.5, 85.0, 90.0]
False
```

Now appending to `log.readings` leaves the caller's outer list unchanged. For the numeric readings shown here, the elements are immutable numbers, so this is a suitable way to preserve the caller's list against the illustrated append operation.

A shallow copy is not a recursive copy

`readings.copy()` makes a new outer list containing references to the same elements. Nested mutable elements would still be shared. The source explicitly requires a deliberate copying policy; it does not promise universal independence for arbitrary nested structures.

A useful documentation sentence says whether the class borrows the caller's list or keeps an outer copy. Without that information, a caller cannot know whether later appends or replacements will be visible through the object.

4.4. Worked answer: an ideal resistor

Lecture-slide alignment: Week 8 source PDF, p. 82. PDF page numbers include title and divider pages.

The optional answer develops the resistor record into a calculation object. It adds the positivity check shown in the source and a method using Ohm's law for an ideal linear resistor.

```python
# Week 8, source PDF page 82: the short worked resistor solution.
class Resistor:
    """Represent an ideal linear resistor; assume finite numeric inputs."""

    def __init__(self, resistance_ohm):
        # This is a positivity check, not a full numeric type validator.
        if resistance_ohm <= 0:
            raise ValueError("Resistance must be positive")
        self.resistance_ohm = resistance_ohm  # Store resistance in ohms.

    def current_a(self, voltage_v):
        # Ohm's law for an ideal linear resistor: I = V / R.
        # A signed voltage produces signed current in the chosen direction.
        return voltage_v / self.resistance_ohm
```

Code 4.6: The source ideal-resistor model

Complete definition in `examples/resistor.py`.

The resistance is persistent state. The voltage is supplied for one calculation. Under the stated model, $I = V/R$, so $12/4 = 3$ A and $0/4 = 0$ A. The method returns current and does not change resistance.

Slide example. Source PDF page(s) 82.

```python
# ENG 101 Week 8: The source worked resistor answer
# Basis: source PDF page(s) 82; slide example.
from resistor import Resistor    # Short model assumes finite numeric inputs.

r = Resistor(4.0)                # Store resistance in ohms on this instance.
print(r.current_a(12.0))         # Hand check: 12 V / 4 ohm = 3 A.
print(r.current_a(0.0))          # Hand check: zero voltage gives zero current.
assert r.current_a(12.0) == 3.0
assert r.current_a(0.0) == 0.0
```

Code 4.7: The source worked resistor answer

Example file: `ex39_resistor_calculation.py`

```
3.0
0.0
```

The positivity check rejects zero and negative resistance with `ValueError`. The following worked test makes both selected failures observable.

Worked pair-activity test. Source PDF page(s) 74, 82.

```python
# ENG 101 Week 8: A failure test for the short resistor
# Basis: source PDF page(s) 74, 82; worked pair-activity test.
from resistor import Resistor  # The source class explicitly checks positivity.

# Both zero and negative resistance violate this class's stated rule.
for resistance_ohm in [0.0, -4.0]:
    try:
        Resistor(resistance_ohm)
    except ValueError:
        print("Rejected nonpositive resistance")
    else:
        raise AssertionError("Expected nonpositive resistance to fail")
```

Code 4.8: A failure test for the short resistor

Example file: `ex45_negative_resistance_test.py`

```
Rejected nonpositive resistance
Rejected nonpositive resistance
```

Engineering check

The short source solution assumes finite numeric inputs and an ideal linear resistor. It is not a full validation library, and the notes do not silently apply the pressure helper to it. In particular, the pressure model's nonnegative-input rule should not automatically be imposed on signed voltage. A contract must follow the quantity and sign convention it represents.

4.5. Common class mistakes: read the cause, not only the symptom

Lecture-slide alignment: Week 8 source PDF, p. 83; related examples on pp. 14, 26, 34, 45, 50, 52. PDF page numbers include title and divider pages.

Symptom	Likely cause	First check
Missing expected field	Local name instead of attribute, or mismatched attribute names.	Initialization assignment and exact spelling.
Wrong argument count	Missing `self` or missing receiver in the call.	Definition and call together.
Wrong branch without error	Method referenced but not called.	Missing parentheses.
Two names change together	Both refer to one mutable instance.	Identity and aliasing.
Initialization rejects a return	Non-`None` value returned by `__init__`.	Remove the value-return statement.

The following intentional-error files expand listed defects into small independent experiments. They are not reference implementations. Each can be run separately to inspect its exception and then repaired by changing one cause.

4.5.1. An initializer incorrectly returns the instance

Intentional error; run separately. Worked illustration of a listed defect. Source PDF page(s) 14, 83.

```
# ENG 101 Week 8: Intentional non-None initializer return
# Basis: source PDF page(s) 14, 83; worked illustration of a listed defect.
# Worked illustration of the initializer error listed in the slides.
class Sensor:
    def __init__(self, value):
        self.value = value       # The field assignment itself is valid.
        return self              # BUG: __init__ must not return this object.

s = Sensor(82.5)                 # TypeError prevents successful construction.
```

Code 4.9: Intentional non-None initializer return

Example file: `intentional_errors/ex40_init_return.py`

```
TypeError: __init__() should return None, not 'Sensor'
```

Only the final exception line is shown; traceback paths and version-dependent wording are omitted.

The field assignment is not the defect. The incorrect `return self` violates the initializer rule already stated in Lecture 22. Remove that return statement and test that a normal construction call produces an object whose field can be read.

4.5.2. A requested field was stored under another name

Intentional error; run separately. Worked debugging exercise. Source PDF page(s) 83.

```python
# ENG 101 Week 8: Intentional attribute-name mismatch
# Basis: source PDF page(s) 83; worked debugging exercise.
# Worked illustration: the saved and requested attribute names differ.
class Sensor:
    def __init__(self, value):
        self.reading = value     # This creates reading, not value.

s = Sensor(82.5)
print(s.value)                   # Expected AttributeError.
```

Code 4.10: Intentional attribute-name mismatch

Example file: `intentional_errors/ex41_attribute_typo.py`

```
AttributeError: 'Sensor' object has no attribute 'value'
```

Only the final exception line is shown; traceback paths and version-dependent wording are omitted.

The initializer stores `reading`, but the caller requests `value`. Choose one documented attribute name and use it consistently. Creating two separate fields merely to suppress the error can introduce a new consistency problem when only one is later updated.

4.5.3. A number is mistakenly called

Intentional error; run separately. Worked illustration of the slide warning. Source PDF page(s) 34.

```python
# ENG 101 Week 8: Intentional call of a numeric attribute
# Basis: source PDF page(s) 34; worked illustration of the slide warning.
initial_value = 82.5  # A number will be stored, not a function.
class Sensor:
    def __init__(self, value):
        self.value = value       # Store a floating-point value.

s = Sensor(initial_value)
print(s.value())                 # BUG: a float cannot be called.
```

Code 4.11: Intentional call of a numeric attribute

Example file: `intentional_errors/ex42_numeric_not_callable.py`

```
TypeError: 'float' object is not callable
```

Only the final exception line is shown; traceback paths and version-dependent wording are omitted.

The field contains a float: read `s.value`, not `s.value()`. Identify the object's type before changing punctuation. Data attributes and methods have different interfaces.

5 Synthesis and Cumulative Practice

Lecture-slide alignment: Week 8 source PDF, pp. 84–87; worked practice based on the three lectures. PDF page numbers include title and divider pages.

5.1. Connect the ideas before memorizing the syntax

The concept map in the deck starts with a class definition and connects it to instances, data attributes, and methods. Testing surrounds the relationships: check initial state, the meaning of each operation, and the state after every important change.

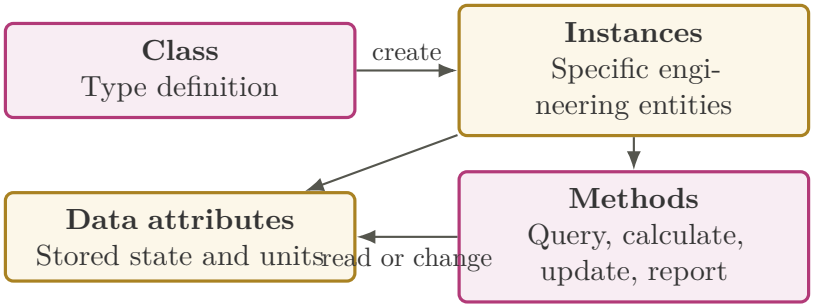


Figure 5.1.: Week 8 synthesis, adapted from source PDF page 84. Testing must cover state as well as behavior.

The four ideas are related but not interchangeable. A class is not one particular reading. A variable naming an instance is not automatically a copy. A method reference is not the result of a call. A returned value is not necessarily a state change. A true interval query is not a certificate of real-system safety.

5.2. Quick reference for reading unfamiliar class code

Pattern	Question to ask
`class Sensor:`	What kind of entity is being defined?
`def __init__(self, ...):`	What state is established on construction?
`self.value = value`	Which input is stored on this receiver?
`s = Sensor(...)`	Which arguments create this instance?
`s.value`	Which current field is read?
`s.report`	Is this a method object, not a report?
`s.report()`	What is returned, and is state unchanged?
`s.update(x)`	What changes, what is rejected, and what is returned?
`alias = s`	Is this another name for the same object?
`a is b`	Do both references identify the same object?
`self.readings = []`	Is a new list created on each initialization?
`self.readings = readings.copy()`	What outer copy is made, and what remains shared?

Use the definition and call together. A method name alone cannot reveal every contract. The short interval sensor assumes valid values; the checked pressure sensor enforces several specific input rules; the ideal-resistor solution checks positivity while retaining other assumptions. Read the body before claiming that a class rejects every bad input.

5.3. Twelve practice questions

These are additional self-study exercises using the slide concepts. They are not Assignment 4 or the official Laboratory 3 brief. Write predictions before looking at the worked answers in the next section.

Q1. In `tank = Sensor("Tank", "deg C", 25.0, 20.0, 30.0)`, identify the class, the instance name, one attribute, and one method. Explain what belongs to the class definition and what belongs to this particular instance.

Q2. An initializer has a parameter named `initial_value`. Explain the difference between assigning `value = initial_value` and `self.value = initial_value`. Which assignment supports a later read of `s.value`?

Q3. The three-field reporting class receives `Sensor("Pipe", "kPa", 315.0)`. Predict the result of `s.report()` and explain why the method definition includes `self` although the call has no explicit argument.

Q4. An out-of-range sensor is tested using `if s.is_safe:`. Why can the wrong branch execute without an exception? State a minimal repair and a test case that would expose the original error.

Q5. A sensor initially stores 82.5 with limits 0.0–90.0. A Boolean is saved from its query, then the reading is updated to 95.0. State the saved Boolean, a fresh query result, the current reading, and the return value of the update.

Q6. Using the illustrative densities 2700 and 7850 kg/m^3, calculate the masses returned for a volume of 0.004 m^3. Does the method need to store this volume on the material? Explain its assumptions.

Q7. The three-sensor list in the slides is filtered for readings outside their intervals. Which sensor is selected? What happens to that selected reference if its value later changes? Does the list automatically re-filter itself?

Q8. Compare `backup = original` with explicitly constructing a new `Sensor` from the original's five fields. Which preserves a snapshot for these numeric/string fields? What qualification is needed for nested mutable data?

Q9. For the checked pressure helper, classify `315`, `"315"`, `True`, `-5.0`, and `float("inf")`. State the returned value or expected exception category for each.

Q10. A pressure sensor starts at 100.0 kPa with limit 400.0 kPa. The caller proposes 450.0, −1.0, and 200.0 in that order, catching documented failures. Trace the stored value and the event reported after each attempt.

Q11. A query and a report promise not to change state. Design a test of that promise for the five-field interval sensor. Why is testing only the returned string insufficient?

Q12. An invalid pressure update leaves the old value unchanged but raises no exception. Has it

fulfilled the source contract? Write the structure of a test that detects this error as well as a state-corrupting failure.

5.4. Worked answers and explanations

Q1–Q3: vocabulary, initialization, and the receiver

Q1. `Sensor` is the class. `tank` is a name referring to one instance. `tank.value` is one attribute, and `tank.is_safe()` calls one method. The class definition specifies the operations and initialization structure. The particular instance stores `"Tank"`, `"deg C"`, 25.0, 20.0, and 30.0.

Q2. `value = initial_value` assigns only a local name. `self.value = initial_value` assigns the field on the receiver, making it available through that instance after the call. The second form supports `s.value`. The spelling of the parameter does not need to match the spelling of the attribute.

Q3. The returned string is `Pipe:  315.0 kPa`. In an ordinary method call, the instance before the dot is automatically supplied as `self`. The equivalent explanatory call is `Sensor.report(s)` for this ordinary method; it is not necessary to use that longer form in normal code.

Q4–Q5: calls, saved answers, and state changes

Q4. Without parentheses, the condition receives a bound method object, which is truthy. The method body is never evaluated. Use `if s.is_safe():`. A reading of 95.0 with limits 0.0 and 90.0 exposes the defect because the actual query should be false. An interior-only test could miss it.

Q5. The saved Boolean is true; the fresh query is false; the current reading is 95.0; and the update returns `None`. These are four separate observations. The saved Boolean is a result from an earlier state, not a live link to a changing query.

Q6: a calculation method uses state and call arguments

The aluminum result is $2700 \times 0.004 = 10.8$ kg. The steel result is $7850 \times 0.004 = 31.4$ kg. Density belongs to the stored material model, whereas volume is an input for this calculation. There is no need to store the supplied part volume on the material in this design. Assume compatible units, finite numeric inputs, and a constant uniform density. The densities remain illustrative.

Q7: selected references are still live objects

Only Battery voltage is selected from the slide dataset, because 10.8 is below its 11.0 V lower limit. The filtered list contains the same object reference as the original list. Updating that object is therefore visible through the filtered list as well.

The following worked consequence uses the source filtering and aliasing rules. Its changed battery value is a practice modification, not an additional source dataset.

Worked consequence of reference filtering. Source PDF page(s) 49-51.

```python
# ENG 101 Week 8: A filtered list is a selection, not a copy
# Basis: source PDF page(s) 49-51; worked consequence of reference filtering.
from sensor_models import Sensor  # The source filter retains references.

battery = Sensor("Battery voltage", "V", 10.8, 11.0, 12.8)
flagged = [battery]               # A selection made while the query is False.
battery.update(12.0)              # The selected object can later change.
print(flagged[0].value)           # The list now exposes the current 12.0.
print(flagged[0].is_safe())       # It is within limits now.
print(len(flagged))              # Selection membership is not recomputed.
```

Code 5.1: A filtered list is a selection, not a copy

Example file: `ex44_snapshot_filter.py`

```
12.0
True
1
```

The selected object now stores 12.0 and its query returns true. However, the list still contains one element. A list does not automatically rerun the Boolean condition that was once used to populate it. Re-filter deliberately when a current selection is required; retain explicit historical data when a historical record is required.

Q8: a snapshot needs a separate object

`backup = original` creates an alias. Constructing a new `Sensor` from the five current fields creates a distinct instance and is sufficient here because its fields are numbers and strings. With nested mutable fields, passing the original field references could still share data. The source therefore requires a deliberate copying policy rather than a blanket claim that every new instance is independent in every respect.

Q9: helper outcomes

Input	Expected outcome
`315`	Return the float `315.0`.
`"315"`	Raise `TypeError`; numeric text is outside this interface.
`True`	Raise `TypeError`; Boolean readings are excluded.
`-5.0`	Raise `ValueError`; negative absolute pressure is rejected.
`float("inf")`	Raise `ValueError`; the result must be finite.

Q10: state after each proposal

Proposal (kPa)	Stored afterward	Event
450.0	450.0	Accepted; limit exceeded.
−1.0	450.0	Rejected with `ValueError`; previous state retained.
200.0	200.0	Accepted; within limit.

The old state retained after the failure is 450.0, the most recently accepted reading, not the initial 100.0.

Q11: non-mutation is observable behavior

A test can save the five scalar/string fields before the calls, execute the query and report, then compare the fields afterward. This is a worked test of the non-mutation promises on source pages 43 and 77.

Worked contract-test extension. Source PDF page(s) 43, 71, 77.

```python
# ENG 101 Week 8: Check that queries and reports preserve state
# Basis: source PDF page(s) 43, 71, 77; worked contract-test extension.
from sensor_models import Sensor  # Query and report should not update state.

s = Sensor("Test", "deg C", 82.5, 0.0, 90.0)
before = (s.name, s.unit, s.value, s.lower_limit, s.upper_limit)
answer = s.is_safe()              # Calculate a Boolean, not a state change.
message = s.report()              # Construct text from the same state.
after = (s.name, s.unit, s.value, s.lower_limit, s.upper_limit)
assert before == after           # Test the promised absence of mutation.
assert answer is True
assert isinstance(message, str)
print("Query and report preserved the state")
```

Code 5.2: Check that queries and reports preserve state

Example file: `ex43_nonmutating_queries.py`

```
Query and report preserved the state
```

The returned Boolean and string are also checked, but neither alone proves that state was preserved. A method could return the expected text and still accidentally change a limit or reading. The before/after comparison targets that additional possible defect.

Q12: required rejection and preserved state

The silently ignored update has not fulfilled the contract. The source requires an exception as well as an unchanged value. Example `ex30` supplies the complete test: save the old reading; attempt the invalid update in `try`; accept only the expected `ValueError`; use `else` to raise a failure if no error occurred; then assert the old state is still present.

A test that checks only the exception can miss corruption before the exception. A test that checks only preserved state can miss silent ignoring. The two observations are complementary.

5.5. Readiness checklist and the next scope boundary

Before moving on, check that you can draw two instances, identify each method's receiver, distinguish `value` from `self.value`, and trace a result separately from a state change. Diagnose missing calls, test boundaries and failed updates, and explain when a function or dictionary is simpler than a class.

Session 9 adds controlled access, class variables, and inheritance. Keep the same habits: *keep data meaningful, make behavior explicit, and test every important change.*

6 Glossary and Revision Checklist

6.1. Week 8 glossary

Term	Meaning in these notes
Object-oriented programming	Organizing a program around entities whose state and related operations are made explicit through objects.
Class	A definition of an object type and its operations, such as `Sensor`.
Instance	One particular object created from a class. Distinct instances can have the same type and different stored values.
Instance name	A variable referring to an instance. Another name can refer to the same instance without creating a new object.
State	The information currently stored by an object, such as a reading and its limits.
Behavior	What an object can calculate, answer, change, or report through its operations.
Data attribute	A named value associated with an instance, such as `s.value`.
Method	An operation accessed through an object. The ordinary instance methods studied here receive the selected instance as their first parameter.
Receiver	The instance selected for a method call, normally the expression before the dot.
`self`	The conventional first parameter name for the receiver of an ordinary instance method; not a Python keyword.
Initialization	Establishing a new instance's expected initial state.
`__init__`	The special initialization method invoked during ordinary instance construction. It must not return a non-`None` value.
Construction call	A class call such as `Sensor(...)` that creates and initializes an instance in these examples.
Dot notation	Attribute access such as `s.value` or method access such as `s.report`. Parentheses are needed to call a method.
Local parameter	A name receiving an argument for one call. It is not automatically an attribute unless stored on the instance.
Bound method	A method object associated with a receiver; obtaining it is not the same as calling it.
Query method	An operation returning an answer about state without changing that state in the stated design.
Calculation method	An operation combining stored information and call arguments to calculate a result.
Update method	An operation intended to change an instance's state. In the source models, successful updates return `None`.
Report method	An operation returning human-readable text in this teaching package. It does not print internally.

Term	Meaning in these notes
Interface	The visible agreement about method names, arguments, returned values, changes, and failures.
Contract	Assumptions and guarantees describing valid use and expected behavior.
Inclusive boundary	A limit accepted at equality, implemented here using `<=`.
Input validity	Whether a proposed value belongs to the input domain of the model.
Operating criterion	A rule for interpreting a valid reading, distinct from input validation.
Finite value	A numeric value that is neither infinity nor NaN in the helper's floating-point representation.
NaN	A special floating-point value meaning "not a number"; rejected by this model's finiteness check.
Committed update	A state assignment performed after the required checks succeed.
Failed-update guarantee	A documented rejection leaves the previous stored state unchanged.
Public attribute	An attribute that the caller can assign directly in the plain classes shown here; such assignment can bypass a checked method.
Alias	A second reference to the same object, not an independent copy.
Identity	Which particular object a reference selects; compared with `is` or `is not`.
Snapshot	A deliberately preserved record of a previous state; creating one requires an appropriate copying policy.
Shallow copy	A new outer container referencing the same elements; nested mutable elements are not recursively copied.
Metadata	Context such as names, units, and limits that gives a numeric reading meaning.
Non-mutation test	A comparison showing that an operation promised not to change state has left the selected fields unchanged.
Independent-instance test	A check that updating one separately created instance does not update another.
Expected-failure test	A check that a specified operation raises the documented error; for stateful objects, also check the state afterward.
Regression test	A retained test of a previously discovered defect, run again after later changes.

6.2. A one-page review procedure

Read the responsibility. Name the entity and state the job of the class. Reject vague claims such as "this class handles everything about sensors." List excluded responsibilities when they could otherwise be assumed.

Inspect construction. Identify the arguments, the initializer's local parameters, and the attributes assigned on the receiver. Ask which assumptions are merely documented and which are actually checked.

Inspect methods. For each method, list the fields read, fields changed, returned value, and failure behavior. Distinguish `s.method` from `s.method()`. Check whether the operation should be a query, update, calculation, or report.

Inspect ownership. Draw names and object references. Do not interpret assignment as automatic copying. Trace mutable list attributes and state which copies, if any, are made.

Inspect evidence. Include initialization, known answers, exact boundaries, above-limit valid inputs, rejected proposals, and a second instance. Check both an expected exception and preserved state. Keep test expectations independent of the implementation under test.

Inspect meaning. Confirm the unit convention and the distinction between valid data and an operating limit. Do not equate a method name or Boolean result with certified engineering safety.

A Sources, Clarifications, and Slide Coverage

A.1. Source basis and suggested reading

The attached Week 8 slide PDF is the controlling source for these reading notes. It defines the three lectures, the examples, the activity sequence, the reference topics, and the boundary with Session 9. The source's suggested reading on PDF page 85 is retained below as a guide, not presented as additional assigned material invented for this document.

The deck refers to the ENG 101 Course Specification for Session 8 and the Session 9 boundary; Wang's supplied *Programming for Engineers with Python*, Sections 15.1–15.11, 15.15–15.17, and 15.26–15.27; and Guttag's second edition, Section 8.1, with inheritance deferred. Its list-ownership reference also points to Wang's Section 5.10 and Appendix H.

The official Python documentation was consulted only for narrow technical checks already relevant to the deck: ordinary receiver binding, initialization return behavior, the subclass-aware meaning of `isinstance`, and `math.isfinite`. Those checks do not change the course scope or silently replace the source's examples.

A.2. Adaptation and clarification record

Lecture-slide alignment: Week 8 source PDF, p. 86, with cross-references to the relevant examples. PDF page numbers include title and divider pages.

Source behavior retained. The successive three-field and five-field sensor definitions are kept distinct. Reports return text rather than printing internally. The source method name `is_safe` is retained, while prose and output explain the narrower within-limit meaning. `PressureSensor` remains independent of `Sensor`; inheritance, class variables, and detailed information hiding are not introduced.

Code presentation. Comments and docstrings explain the operations. Source PDF line wraps are reformatted into valid Python lines. Related class fragments are assembled into complete definitions without changing their tests or return behavior. Prerequisite imports reuse the matching annotated model files. The numeric helper includes the required `math` import.

Worked additions. Material and resistor activity answers, receiver repairs, observed outputs, before/after test examples, and cumulative exercises are labelled as worked solutions or extensions. The calls added to demonstrate the optional `ReadingLog` constructors are not claimed to be printed on the original slides. The lab solution's initial 25.0-degree setup value is explicitly an illustrative choice; the five processed readings and stated results come from source page 76.

Failed-update testing. Source page 66 is preserved as a demonstration. The stronger test in Example `ex30` explicitly requires the exception, addressing the testing gap identified by source page 67. This strengthening is shown separately rather than silently inserted into the source demonstration.

Implementation qualifications. The numeric helper's `isinstance` check accepts suitable subclasses despite the source's built-in-only wording. The source initializer uses `ValueError` for a wrong-type name as well as blank text. The short resistor example does not implement a

complete type/finiteness check. These behaviors are made explicit rather than replaced.

State and reporting limits. The plain checked class still permits direct public-attribute assignment. A labelled misuse example demonstrates that bypass; it is excluded from the working combined demonstration. The generic interval report omits the numeric limits, while the pressure report includes its threshold. Reference filtering retains live objects and is not an automatically refreshed classification list.

Engineering and assessment scope. Densities, limits, and practice inputs are illustrative. A within-limit result is not certified real-system safety. The notes add no hardware acquisition, control action, official assessment requirements, or new grading rules.

A.3. Complete source-page coverage map

Every page of the supplied slide PDF appears in the map below. Locations point to reading-note sections, not to slide-footer numbers. A source page with an overview, divider, learning goal, or closing statement is mapped to the corresponding introduction or synthesis.

PDF p.	Source topic / title	Reading-note location
1	Programming for Engineers	About, p. 2
2	Week 8 at a glance	Study guide, p. 3
3	Course scope: a first step into OOP	Study guide, p. 3
4	LECTURE 22: Objects, Classes, and Data Attributes	Section 1.1, p. 7
5	Lecture 22 learning goals	Section 1.1, p. 7
6	Warm-up: a familiar sensor record	Section 1.2, p. 7
7	Why introduce another way to organize code?	Section 1.3, p. 8
8	You have already used objects and methods	Section 1.4, p. 8
9	Four words that must stay distinct	Section 1.5, p. 9
10	One definition, several distinct instances	Section 1.6, p. 10
11	The smallest class: syntax before usefulness	Section 1.7, p. 10
12	Manual fields can leave incomplete objects	Section 1.7, p. 10
13	Initialize the expected fields in one place	Section 1.8, p. 11
14	Creating an instance is a sequence of events	Section 1.8, p. 11
15	The two sides of self.value = value	Section 1.9, p. 12
16	self means this particular instance	Section 1.9, p. 12
17	Create and inspect two sensors	Section 1.10, p. 13
18	Attributes are accessed with a dot	Section 1.10, p. 13
19	A local value is not a live link to the sensor	Section 1.11, p. 14
20	Trace the state after each assignment	Section 1.11, p. 14
21	Inspect the type, not only the printed value	Section 1.12, p. 15
22	A dictionary and a class are both valid tools	Section 1.13, p. 15
23	Advantages come with responsibilities	Section 1.13, p. 15
24	Live coding: a material data model	Section 1.14, p. 16
25	Test an instance immediately after creation	Section 1.15, p. 17
26	Debugging clinic: local name or attribute?	Section 1.16, p. 17
27	A first-class design checklist	Section 1.17, p. 19
28	Lecture 22 exit ticket	Section 1.18, p. 19
29	LECTURE 23: Methods and Object Behavior	Section 2.1, p. 20
30	Lecture 23 learning goals	Section 2.1, p. 20

PDF p.	Source topic / title	Reading-note location
31	Warm-up: identify the receiver	Section 2.1, p. 20
32	A method uses its receiver's data	Section 2.2, p. 20
33	Why does the call omit self?	Section 2.3, p. 21
34	Read data; call behavior	Section 2.4, p. 22
35	Give the sensor a clear interval contract	Section 2.5, p. 22
36	Initialize an instance with its own limits	Section 2.5, p. 22
37	A query method answers a Boolean question	Section 2.6, p. 23
38	An update method changes state deliberately	Section 2.6, p. 23
39	A report can call a query method	Section 2.6, p. 23
40	Return and print are still different	Section 2.7, p. 24
41	Test both boundaries, not one convenient value	Section 2.8, p. 25
42	Trace a reading before and after an update	Section 2.8, p. 25
43	Four kinds of method, four different contracts	Section 2.9, p. 26
44	Prediction: old result or current state?	Section 2.10, p. 27
45	A method reference is not a decision result	Section 2.11, p. 27
46	A material can calculate with its stored density	Section 2.12, p. 29
47	Check the calculation independently	Section 2.12, p. 29
48	Objects fit inside familiar lists	Section 2.13, p. 30
49	Filter objects without losing their metadata	Section 2.14, p. 30
50	Aliasing did not disappear when we made a class	Section 2.15, p. 31
51	A new instance can preserve a snapshot	Section 2.15, p. 31
52	Debugging clinic: find the receiver mistakes	Section 2.16, p. 33
53	Lecture 23 exit ticket	Section 2.17, p. 34
54	LECTURE 24: Reliable Engineering Objects	Section 3.1, p. 35
55	Lecture 24 learning goals	Section 3.1, p. 35
56	Warm-up: invalid or outside the operating limit?	Section 3.2, p. 35
57	Decide the responsibility before the methods	Section 3.3, p. 36
58	Write the input and state contract	Section 3.4, p. 36
59	Validate before committing a state change	Section 3.5, p. 37
60	Reuse a checked-number function	Section 3.6, p. 37
61	Initialize a checked pressure object	Section 3.7, p. 38
62	The checked update is still a small method	Section 3.7, p. 38
63	A query should report the current state	Section 3.7, p. 38
64	A high reading is not a parsing error	Section 3.8, p. 39
65	Test a normal value and an exact threshold	Section 3.9, p. 40
66	Test the state after a failed update	Section 3.10, p. 40
67	Use an expected-failure test, not silence	Section 3.10, p. 40
68	Test that separate instances stay separate	Section 3.11, p. 42
69	Combine a method with familiar exception handling	Section 3.12, p. 43
70	Trace the complete monitoring sequence	Section 3.12, p. 43
71	Objects still need a test matrix	Section 3.13, p. 44
72	Public attributes are not automatically protected	Section 3.14, p. 45
73	When is a class worth using?	Section 3.15, p. 46
74	Pair design activity: one entity, clear contracts	Section 3.15, p. 46
75	Assignment 4: prepare to work with objects	Section 3.16, p. 46
76	Laboratory 3: a small practice workflow	Section 3.16, p. 46
77	Review the design, not only the output	Section 3.17, p. 48

PDF p.	Source topic / title	Reading-note location
78	Lecture 24 exit ticket	Section 3.17, p. 48
79	Reference: the complete numeric helper	Section 4.1, p. 49
80	Reference: each instance can own a list	Section 4.2, p. 51
81	Reference: receiving a list is not copying it	Section 4.3, p. 52
82	Worked answer: an ideal resistor class	Section 4.4, p. 54
83	Reference: common class mistakes	Section 4.5, p. 56
84	Week 8 concept map	Section 5.1, p. 58
85	Suggested reading and source basis	Section A.1, p. 66
86	Source adaptations and scope notes	Section A.2, p. 66
87	Objects organize engineering meaning.	Section 5.5, p. 62

Bibliography

[1] Y. Wang, *Programming for Engineers: Week 8 — Objects, Classes, Attributes, and Methods*, School of Engineering, Nazarbayev University, Fall 2026. Supplied 87-page PDF, uploaded as `week8_full(1).pdf`. Primary basis for this document.

[2] Nazarbayev University, *ENG 101 Course Specification Report*, Session 8 and Session 9, as cited in the Week 8 presentation. Session 8: advantages of OOP, attributes, methods, instances, Assignment 4, and Laboratory 3.

[3] Y. Wang, *Programming for Engineers with Python*, supplied working manuscript. Suggested sections reproduced from the Week 8 slide reading list; the attached slides determine the treatment here.

[4] J. V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, second edition, MIT Press, Section 8.1. Suggested reading reproduced from the source deck.

[5] Python Software Foundation, *The Python Tutorial: Classes*, Sections 9.3–9.4. https://docs.python.org/3/tutorial/classes.html. Consulted 14 September 2026 for receiver-binding terminology.

[6] Python Software Foundation, *Data Model*, `object.__init__`. https://docs.python.org/3/reference/datamodel.html#object.__init__. Consulted 14 September 2026 for the non-`None` return restriction.

[7] Python Software Foundation, *Built-in Functions*, `isinstance`. https://docs.python.org/3/library/functions.html#isinstance. Consulted 14 September 2026 for the subclass qualification.

[8] Python Software Foundation, *Mathematical Functions*, `math.isfinite`. https://docs.python.org/3/library/math.html#math.isfinite. Consulted 14 September 2026 for finite-value checking.